	ORIGINATE DATE	EDIT DATE	DOCUMENT-REV. NUM.	PAGE
	9 July, 2003	4 September, 2015	GEN-CXXXXX-REVA	1 of 54

Author: Laurent Lefebvre

Issue To:	**Copy No:**

R400 Sequencer Specification

SQ

Version 2.11

Overview: This is an architectural specification for the R400 Sequencer block (SEQ). It provides an overview of the required capabilities and expected uses of the block. It also describes the block interfaces, internal sub-blocks, and provides internal state diagrams.

AUTOMATICALLY UPDATED FIELDS:
Document Location: C:\perforce\r400\doc_lib\design\blocks\sq\R400_Sequencer.doc
Current Intranet Search Title: R400 Sequencer Specification

APPROVALS	
Name/Dept	Signature/Date

Remarks:

Table Of Contents

Revision Changes:

Rev 0.1 (Laurent Lefebvre) Date: May 7, 2001	First draft.
Rev 0.2 (Laurent Lefebvre) Date : July 9, 2001	Changed the interfaces to reflect the changes in the SP. Added some details in the arbitration section.
Rev 0.3 (Laurent Lefebvre) Date : August 6, 2001	Reviewed the Sequencer spec after the meeting on August 3, 2001.
Rev 0.4 (Laurent Lefebvre) Date : August 24, 2001	Added the dynamic allocation method for register file and an example (written in part by Vic) of the flow of pixels/vertices in the sequencer.
Rev 0.5 (Laurent Lefebvre) Date : September 7, 2001	Added timing diagrams (Vic)
Rev 0.6 (Laurent Lefebvre) Date : September 24, 2001	Changed the spec to reflect the new R400 architecture. Added interfaces.
Rev 0.7 (Laurent Lefebvre) Date : October 5, 2001	Added constant store management, instruction store management, control flow management and data dependant predication.
Rev 0.8 (Laurent Lefebvre) Date : October 8, 2001	Changed the control flow method to be more flexible. Also updated the external interfaces.
Rev 0.9 (Laurent Lefebvre) Date : October 17, 2001	Incorporated changes made in the 10/18/01 control flow meeting. Added a NOP instruction, removed the conditional_execute_or_jump. Added debug registers.
Rev 1.0 (Laurent Lefebvre) Date : October 19, 2001	Refined interfaces to RB. Added state registers.
Rev 1.1 (Laurent Lefebvre) Date : October 26, 2001	Added SEQ→SP0 interfaces. Changed delta precision. Changed VGT→SP0 interface. Debug Methods added.
Rev 1.2 (Laurent Lefebvre) Date : November 16, 2001	Interfaces greatly refined. Cleaned up the spec.
Rev 1.3 (Laurent Lefebvre) Date : November 26, 2001	Added the different interpolation modes.
Rev 1.4 (Laurent Lefebvre) Date : December 6, 2001	Added the auto incrementing counters. Changed the VGT→SQ interface. Added content on constant management. Updated GPRs.
Rev 1.5 (Laurent Lefebvre) Date : December 11, 2001	Removed from the spec all interfaces that weren't directly tied to the SQ. Added explanations on constant management. Added PA→SQ synchronization fields and explanation.
Rev 1.6 (Laurent Lefebvre) Date : January 7, 2002	Added more details on the staging register. Added detail about the parameter caches. Changed the call instruction to a Conditionnal_call instruction. Added details on constant management and updated the diagram.
Rev 1.7 (Laurent Lefebvre) Date : February 4, 2002	Added Real Time parameter control in the SX interface. Updated the control flow section.
Rev 1.8 (Laurent Lefebvre) Date : March 4, 2002	New interfaces to the SX block. Added the end of clause modifier, removed the end of clause instructions.
Rev 1.9 (Laurent Lefebvre) Date : March 18, 2002	Rearangement of the CF instruction bits in order to ensure byte alignement.
Rev 1.10 (Laurent Lefebvre) Date : March 25, 2002	Updated the interfaces and added a section on exporting rules.
Rev 1.11 (Laurent Lefebvre) Date : April 19, 2002	Added CP state report interface. Last version of the spec with the old control flow scheme
Rev 2.0 (Laurent Lefebvre) Date : April 19, 2002	New control flow scheme

Rev 2.01 (Laurent Lefebvre) Date : May 2, 2002	Changed slightly the control flow instructions to allow force jumps and calls.
Rev 2.02 (Laurent Lefebvre) Date : May 13, 2002	Updated the Opcodes. Added type field to the constant/pred interface. Added Last field to the SQ→SP instruction load interface.
Rev 2.03 (Laurent Lefebvre) Date : July 15, 2002	SP interface updated to include predication optimizations. Added the predicate no stall instructions,
Rev 2.04 (Laurent Lefebvre) Date :August 2, 2002	Documented the new parameter generation scheme for XY coordinates points and lines STs.
Rev 2.05 (Laurent Lefebvre) Date : September 10, 2002	Some interface changes and an architectural change to the auto-counter scheme.
Rev 2.06 (Laurent Lefebvre) Date : October 11, 2002	Widened the event interface to 5 bits. Some other little typos corrected.
Rev 2.07 (Laurent Lefebvre) Date : October 14, 2002	Loops, jumps and calls are now using a 13 bit address which allows to jump and call and loop around any control flow addresses (does not requires to be even anymore).
Rev 2.08 (Laurent Lefebvre) Date : October 16, 2002	Clarification updates after discussion with Clay.
Rev 2.09 (Laurent Lefebvre) Date : January 7, 2003	Corrected the SQ→SP staging register interface.
Rev 2.10 (Laurent Lefebvre) Date : April 8, 2003	Adding R500 modifications
Rev 2.11 (Laurent Lefebvre) Date : May 1, 2003	Adding SQ->SP updated interfaces

1. Overview

The sequencer chooses four ALU threads (two from each bank), a vertex cache and a fetch thread to execute, and executes all of the instructions in a block before looking for a new clause of the same type. Two ALU threads are executed interleaved to hide the ALU latency. The arbitrator will give priority to older threads. There are two separate reservation stations, one for pixel vectors and one for vertices vectors. This way a pixel can pass a vertex and a vertex can pass a pixel.

There are also 2 separate ALU banks from which the SQ picks the ALU threads to be executed in parallel.

To support the shader pipe the sequencer also contains the shader instruction store, constant store, control flow constants and texture state. The height shader pipes also execute the same two instructions thus there is only one sequencer for the whole chip but it issues 2 instructions every four clocks.

The sequencer first arbitrates between vectors of 64 vertices that arrive directly from primitive assembly and vectors of 16 quads (64 pixels) that are generated in the scan converter.

The vertex or pixel program specifies how many GPRs it needs to execute. The sequencer will not start the next vector until the needed space is available in the GPRs.

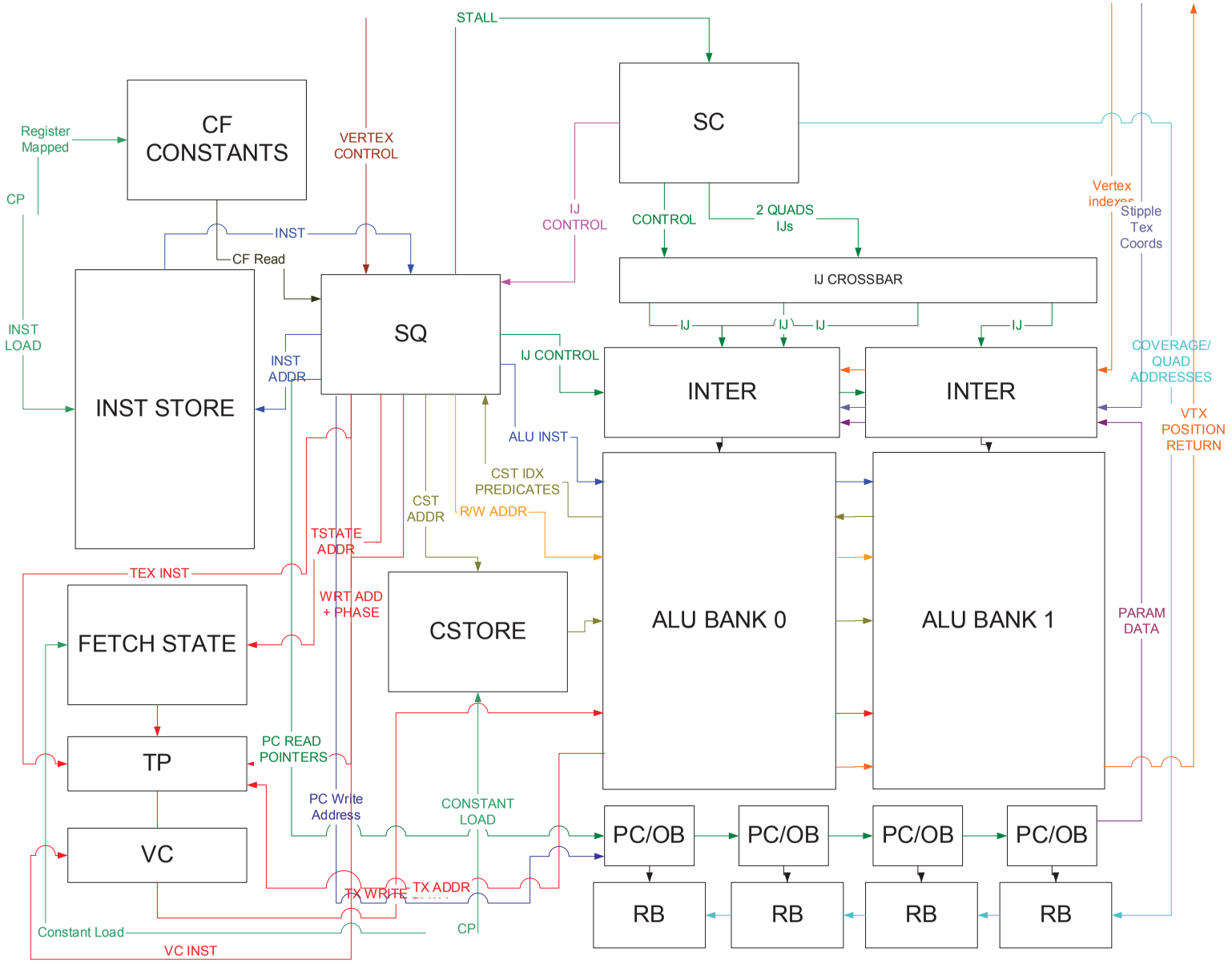


Figure 1: General Sequencer overview

1.1 Top Level Block Diagram

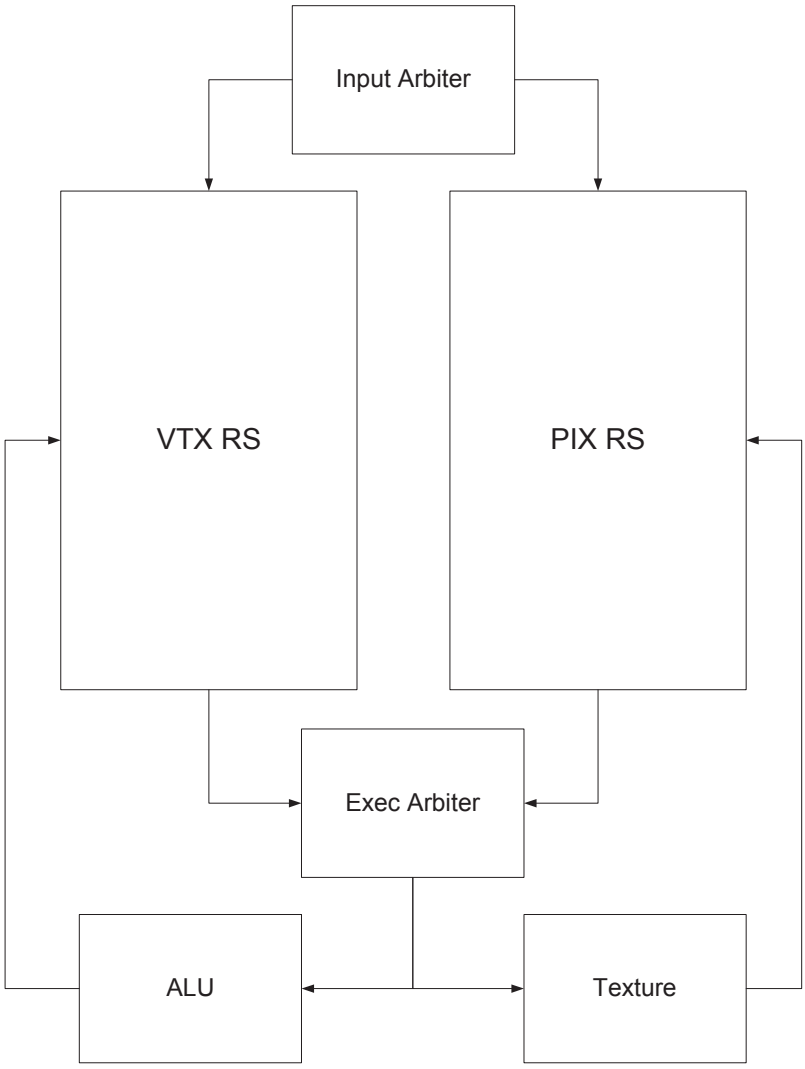


Figure 2: Reservation stations and arbiters

Under this new scheme, the sequencer (SQ) will only use one global state management machine per vector type (pixel, vertex) that we call the reservation station (RS).

1.2 Data Flow graph (SP)

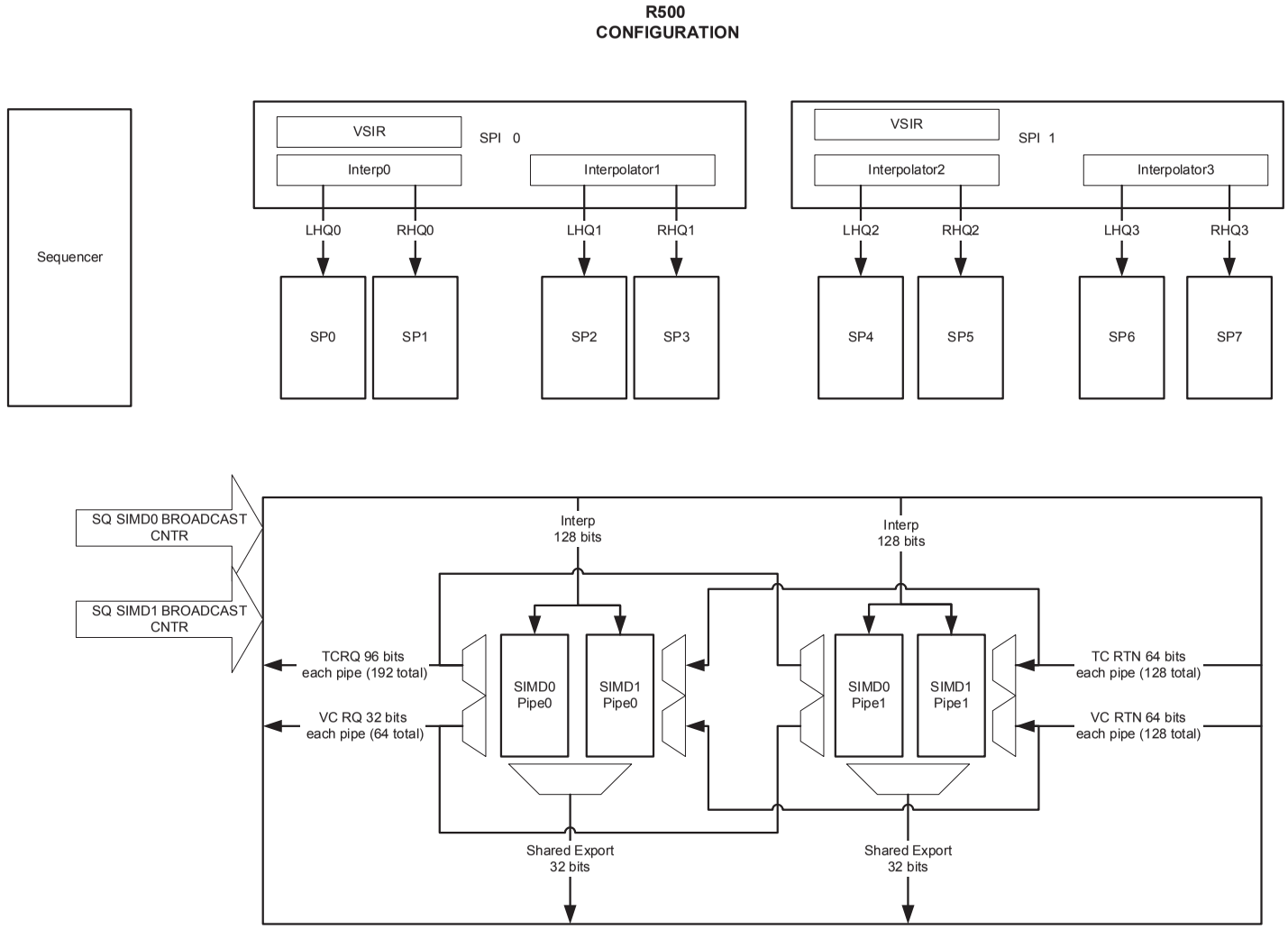


Figure 3: The shader Pipe

1.3 Control Graph

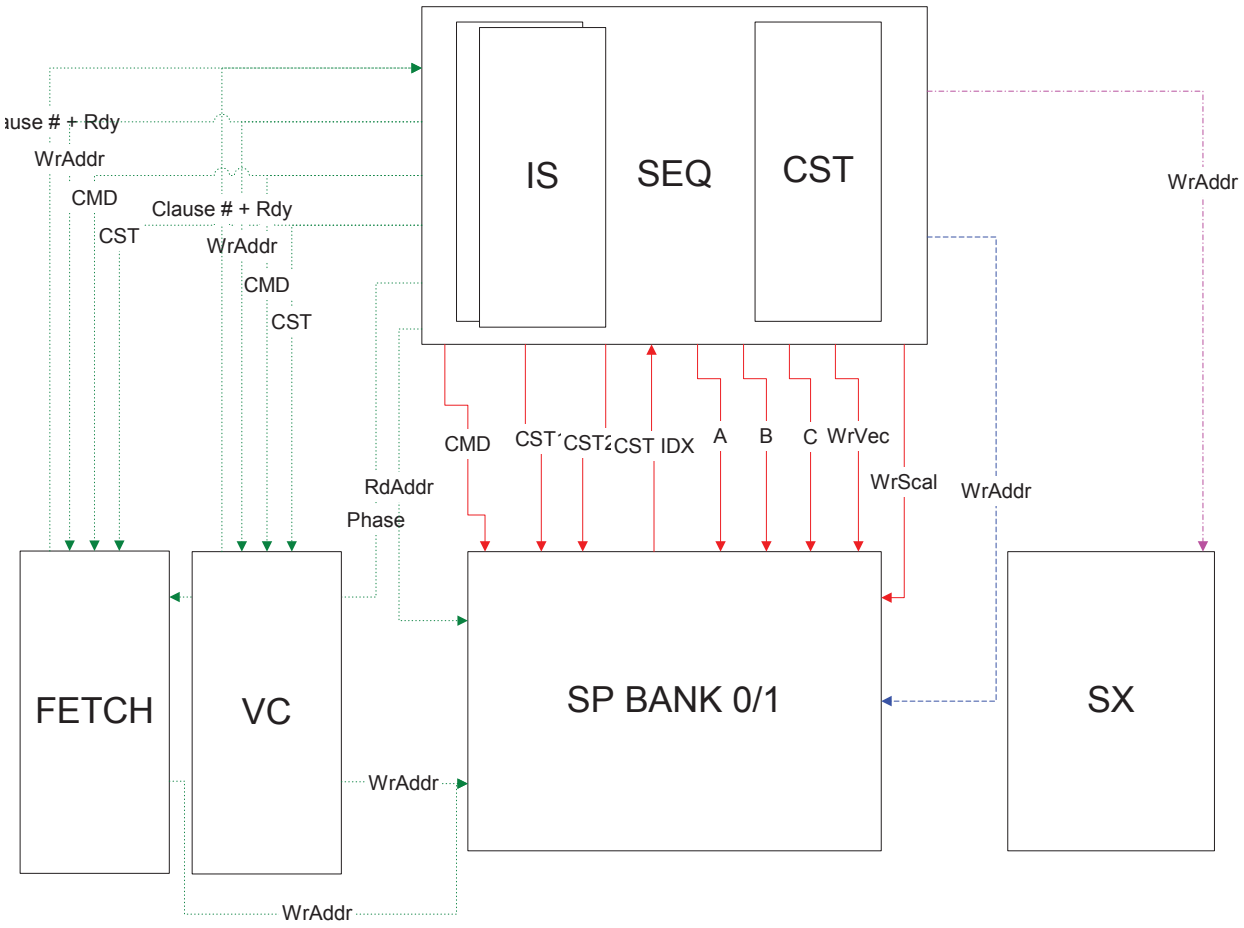


Figure 4: Sequencer Control interfaces

In green is represented the Fetch control interface, in red the ALU control interface, in blue the Interpolated/Vector control interface and in purple is the output file control interface.

2. Interpolated data bus

The interpolators contain an IJ buffer to pack the information as much as possible before writing it to the register file.

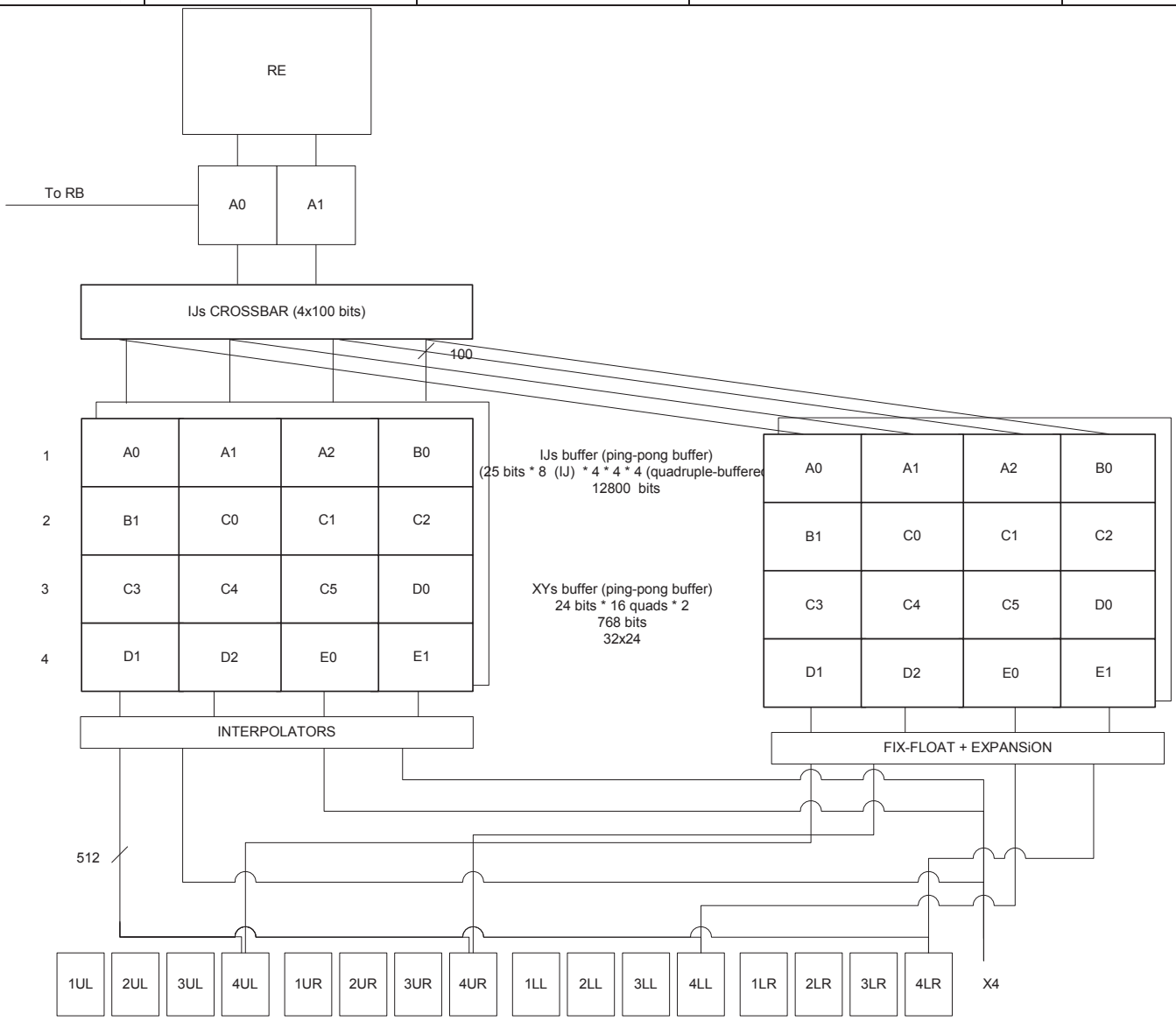


Figure 5: Interpolation buffers

	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9	T10	T11	T12	T13	T14	T15	T16	T17	T18	T19	T20	T21	T22	T23
											WRITES													
SP 0	A0	A0	XY A0	B1	B1	XY B1	C3	C3	XY C3				D1	D1	XY D1									
SP 1	A1	A1	XY A1				C0	C0	XY C0	C4	C4	XY C4	D2	D2	XY D2									
SP 2	A2	A2	XY A2				C1	C1	XY C1	C5	C5	XY C5				E0	E0	XY E0						
SP 3				B0	B0	XY B0	C2	C2	XY C2				D0	D0	XY D0	E1	E1	XY E1						
											READS													
SP 0	XY 0-3	XY 16-19	XY 32-35	XY 48-51	A0	B1	C3	D1					A0	B1	C3	D1					V 0-3	V 16-19	V 32-35	V 48-51
SP 1	XY 4-7	XY 20-23	XY 36-39	XY 52-55	A1		C4	D2		C0			A1		C4	D2		C0			V 4-7	V 20-23	V 36-39	V 52-55
SP 2	XY 8-11	XY 24-27	XY 40-43	XY 56-59	A2		C5			C1		E0	A2		C5			C1		E0	V 8-11	V 24-27	V 40-43	V 56-59
SP 3	XY 12-15	XY 28-31	XY 44-47	XY 60-63					B0	C2	D0	E1					B0	C2	D0	E1	V 12-15	V 28-31	V 44-47	V 60-63
		XY						P1								P2					VTX			

Figure 6: Interpolation timing diagram

Above is an example of a tile the sequencer might receive from the SC. The write side is how the data get stacked into the XY and IJ buffers, the read side is how the data is passed to the GPRs. The IJ information is packed in the IJ buffer 4 quads at a time or two clocks. The sequencer allows at any given time as many as four quads to interpolate a parameter. They all have to come from the same primitive. Then the sequencer controls the write mask to the GPRs to write the valid data in.

3. Instruction Store

There is going to be two instruction stores for the whole chip. They will each contain 4096 instructions of 96 bits each.

They will be 1 port memories; Ports are allocated in this fashion (but not necessarily in this order):

ALU 0 SIMD0 CF
ALU 0 SIMD0
ALU 1 SIMD0 CF
ALU 1 SIMD0
Fetch CF
Fetch
VC CF
VC

ALU 0 SIMD1 CF
ALU 0 SIMD1
ALU 1 SIMD1 CF
ALU 1 SIMD1
Fetch CF
Fetch
VC CF
VC

Fetch and VC can steal one another's ports with stated resource having priority over its port (this is not really necessary for the R500 but will be for any derivative part because there will only be one instruction store).

Writes are opportunistic.

The instruction store is loaded by the CP thru the register mapped registers.

The VS_BASE and PS_BASE context registers are used to specify for each context where its shader is in the instruction memory.

For the Real time commands the story is quite the same but for some small differences. There are no wrap-around points for real time so the driver must be careful not to overwrite regular shader data. The shared code (shared subroutines) uses the same path as real time.

4. Sequencer Instructions

All control flow instructions instructions are handled by the sequencer only. The ALUs will perform NOPs during this time (MOV PV,PV, PS,PS) if they have nothing else to do.

5. Constant Stores

5.1 Memory organizations

A likely size for the ALU constant store is 1024x128 bits. The read BW from the ALU constant store is 128 bits/clock and the write bandwidth is 32 bits/clock (directed by the CP bus size not by memory ports).

The maximum logical size of the constant store for a given shader is 256 constants. Or 512 for the pixel/vertex shader pair. The size of the re-mapping table is 128 lines (each line addresses 4 constants). The write granularity is 4 constants or 512 bits. It takes 16 clocks to write the four constants. Real time requires 256 lines in the physical memory (this is physically register mapped).

There will be two of those memories and two of each remapping read memories.

The texture state is also kept in a similar memory. The size of this memory is 320x96 bits (128 texture states for regular mode, 32 states for RT). The memory thus holds 128 texture states (192 bits per state). The logical size exposes 32 different states total, which are going to be shared between the pixel and the vertex shader. The size of the re-mapping table to for the texture state memory is 32 lines (each line addresses 1 texture state lines in the real

memory). The CP write granularity is 1 texture state lines (or 192 bits). The driver sends 512 bits but the CP ignores the top 320 bits. It thus takes 6 clocks to write the texture state. Real time requires 32 lines in the physical memory (this is physically register mapped).

The control flow constant memory doesn't sit behind a renaming table. It is register mapped and thus the driver must reload its content each time there is a change in the control flow constants. Its size is 320*32 because it must hold 8 copies of the 32 dwords of control flow constants and the loop construct constants must be aligned.

The constant re-mapping tables for texture state and ALU constants are logically register mapped for regular mode and physically register mapped for RT operation.

5.2 Management of the Control Flow Constants

The control flow constants are register mapped, thus the CP writes to the according register to set the constant, the SQ decodes the address and writes to the block pointed by its current base pointer (CF_WR_BASE). On the read side, one level of indirection is used. A register (SQ_CONTEXT_MISC.CF_RD_BASE) keeps the current base pointer to the control flow block. This register is copied whenever there is a state change. Should the CP write to CF after the state change, the base register is updated with the (current pointer number +1)% number of states. This way, if the CP doesn't write to CF the state is going to use the previous CF constants.

5.3 Management of the re-mapping tables

5.3.1 *R400 Constant management*

The sequencer is responsible to manage two re-mapping tables (one for the constant store and one for the texture state). On a state change (by the driver), the sequencer will broadside copy the contents of its re-mapping tables to a new one. We have 8 different re-mapping tables we can use concurrently.

The constant memory update will be incremental, the driver only need to update the constants that actually changed between the two state changes.

For this model to work in its simplest form, the requirement is that the physical memory MUST be at least twice as large as the logical address space + the space allocated for Real Time. In our case, since the logical address space is 512 and the reserved RT space can be up to 256 entries, the memory must be of sizes 1280 and above. Similarly the size of the texture store must be of 32*2+32 = 96 entries and above.

5.3.2 *Dirty bits*

Two sets of dirty bits will be maintained per logical address. The first one will be set to zero on reset and set when the logical address is addressed. The second one will be set to zero whenever a new context is written and set for each address written while in this context. The reset dirty is not set, then writing to that logical address will not require de-allocation of whatever address stored in the renaming table. If it is set and the context dirty is not set, then the physical address store needs to be de-allocated and a new physical address is necessary to store the incoming data. If they are both set, then the data will be written into the physical address held in the renaming for the current logical address. No de-allocation or allocation takes place. This will happen when the driver does a set constant twice to the same logical address between context changes. NOTE: It is important to detect and prevent this, failure to do it will allow multiple writes to allocate all physical memory and thus hang because a context will not fit for rendering to start and thus free up space.

5.3.3 *Free List Block*

A free list block that would consist of a counter (called the IFC or Initial Free Counter) that would reset to zero and incremented every time a chunk of physical memory is used until they have all been used once. This counter would be checked each time a physical block is needed, and if the original ones have not been used up, us a new one, else check the free list for an available physical block address. The count is the physical address for when getting a chunk from the counter.
Storage of a free list big enough to store all physical block addresses.
Maintain three pointers for the free list that are reset to zero. The first one we will call write_ptr. This pointer will identify the next location to write the physical address of a block to be de-allocated. Note: we can never free more

physical memory locations than we have. Once recording address the pointer will be incremented to walk the free list like a ring.
The second pointer will be called stop_ptr. The stop_ptr pointer will be advanced by the number of address chunks de-allocates when a context finishes. The address between the stop_ptr and write_ptr cannot be reused because they are still in use. But as soon as the context using then is dismissed the stop_ptr will be advanced.
The third pointer will be called read_ptr. This pointer will point will point to the next address that can be used for allocation as long as the read_ptr does not equal the stop_ptr and the IFC is at its maximum count.

5.3.4 *De-allocate Block*

This block will maintain a free physical address block count for each context. While in current context, a count shall be maintained specifying how many blocks were written into the free list at the write_ptr pointer. This count will be reset upon reset or when this context is active on the back and different than the previous context. It is actually a count of blocks in the previous context that will no longer be used. This count will be used to advance the write_ptr pointer to make available the set of physical blocks freed when the previous context was done. This allows the discard or de-allocation of any number of blocks in one clock.

5.3.5 *Operation of Incremental model*

The basic operation of the model would start with the write_ptr, stop_ptr, read_ptr pointers in the free list set to zero and the free list counter is set to zero. Also all the dirty bits and the previous context will be initialized to zero. When the first set constants happen, the reset dirty bit will not be set, so we will allocate a physical location from the free list counter because its not at the max value. The data will be written into physical address zero. Both the additional copy of the renaming table and the context zeros of the big renaming table will be updated for the logical address that was written by set start with physical address of 0. This process will be repeated for any logical address that are not dirty until the context changes. If a logical address is hit that has its dirty bits set while in the same context, both dirty bits would be set, so the new data will be over-written to the last physical address assigned for this logical address. When the first draw command of the context is detected, the previous context stored in the additional renaming table will be copied to the larger renaming table in the current (new) context location. Then the set constant logical address with be loaded with a new physical address during the copy and if the reset dirty was set, the physical address it replaced in the renaming table would be entered at the write_ptr pointer location on the free list and the write_ptr will be incremented. The de-allocation counter for the previous context (eight) will be incremented. This as set states come in for this context one of the following will happen:

1.) No dirty bits are set for the logical address being updated. A line will be allocated of the free-list counter or the free list at read_ptr pointer if read_ptr != to stop_ptr .
2.) Reset dirty set and Context dirty not set. A new physical address is allocated, the physical address in the renaming table is put on the free list at write_ptr and it is incremented along with the de-allocate counter for the last context.
3.) Context dirty is set then the data will be written into the physical address specified by the logical address.

This process will continue as long as set states arrive. This block will provide backpressure to the CP whenever he has not free list entries available (counter at max and stop_ptr == read_ptr). The command stream will keep a count of contexts of constants in use and prevent more than max constants contexts from being sent.

Whenever a draw packet arrives, the content of the re-mapping table is written to the correct re-mapping table for the context number. Also if the next context uses less constants than the current one all exceeding lines are moved to the free list to be de-allocated later. This happens in parallel with the writing of the re-mapping table to the correct memory.

Now preferable when the constant context leaves the last ALU clause it will be sent to this block and compared with the previous context that left. (Init to zero) If they differ than the older context will no longer be referenced and thus can be de-allocated in the physical memory. This is accomplished by adding the number of blocks freed this context to the stop_ptr pointer. This will make all the physical addresses used by this context available to the read_ptr allocate pointer for future allocation.

This device allows representation of multiple contexts of constants data with N copies of the logical address space. It also allows the second context to be represented as the first set plus some new additional data by just storing the delta's. It allows memory to be efficiently used and when the constants updates are small it can store multiple

context. However, if the updates are large, less contexts will be stored and potentially performance will be degraded. Although it will still perform as well as a ring could in this case.

5.4 Constant Store Indexing

In order to do constant store indexing, the sequencer must be loaded first with the indexes (that come from the GPRs). There are 144 wires from the exit of the SP to the sequencer (9 bits pointers x 16 vertexes/clock).

```
MOVA  R1.X,R2.X        // Loads the sequencer with the content of R2.X, also copies the content of R2.X into R1.X
ADD   R3,R4,C0[R2.X]// Uses the state from the sequencer to add R4 to C0[R2.X] into R3
```

Note that we don't really care about what is in the brackets because we use the state from the MOVA instruction. R2.X is just written again for the sake of simplicity and coherency.

The storage needed in the sequencer in order to support this feature is 2*64*9 bits = 1152 bits.

The address register is a signed integer, which ranges from –256 to 255.

The address register is not kept across clause boundaries. As such, it must be refreshed after any Serialize (or yield), allocate instruction or resource change. Failure to refresh the address register will result in unpredictable behavior.

5.5 Real Time Commands

The real time commands constants are written by the CP using the register mapped registers allocated for RT. It works is the same way than when dealing with regular constant loads BUT in this case the CP is not sending a logical address but rather a physical address and the reads are not passing thru the re-mapping table but are directly read from the memory. The boundary between the two zones is defined by the CONST_EO_RT control register. Similarly, for the fetch state, the boundary between the two zones is defined by the TSTATE_EO_RT control register.

5.6 Constant Waterfalling

In order to have a reasonable performance in the case of constant store indexing using the address register, we are going to have the possibility of using the physical memory port for read only. This way we can read 1 constant per clock and thus have a worst-case waterfall mode of 1 vertex per clock. There is a small synchronization issue related with this as we need for the SQ to make sure that the constants where actually written to memory (not only sent to the sequencer) before it can allow the first vector of pixels or vertices of the state to go thru the ALUs. To do so, the sequencer keeps 8 bits (one per render state) and sets the bits whenever the last render state is written to memory and clears the bit whenever a state is freed.

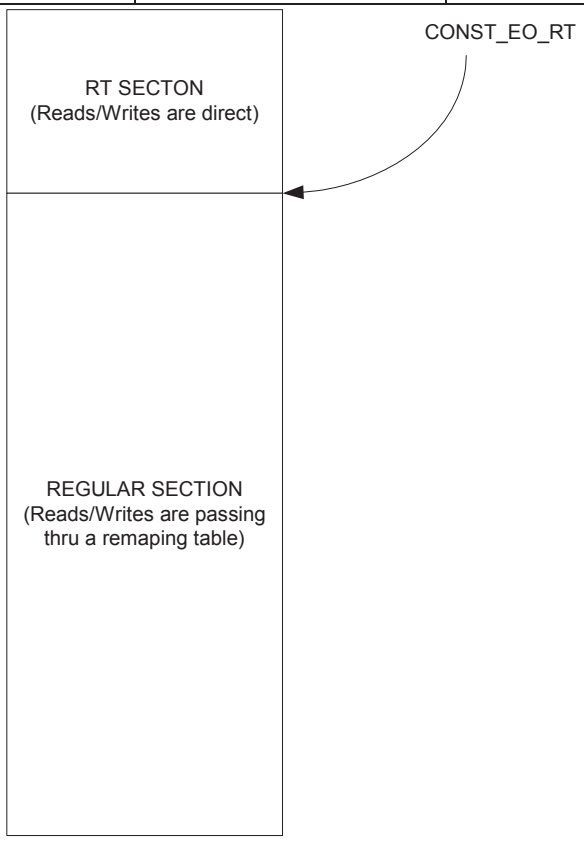


Figure 7: The Constant store

6. Looping and Branches

Loops and branches are planned to be supported and will have to be dealt with at the sequencer level. We plan on supporting constant loops and branches using a control program.

6.1 The controlling state.

The R400 controling state consists of:

Boolean[255:0]
Loop_count[7:0][31:0]
Loop_Start[7:0][31:0]
Loop_Step[7:0][31:0]

That is 256 Booleans and 32 loops.

We have a stack of 4 elements for nested calls of subroutines and 4 loop counters to allow for nested loops.

This state is available on a per shader program basis.

6.2 The Control Flow Program

We'd like to be able to code up a program of the form:

```
1:      Loop
2:      Exec    TexFetch
```

3: TexFetch
4: ALU
5: ALU
6: TexFetch
7: End Loop
8: ALU Export

But realize that 3: may be dependent on 2: and 4: is almost certainly dependent on 2: and 3:. Without clausing, these dependencies need to be expressed in the Control Flow instructions. Additionally, without separate 'texture clauses' and 'ALU clauses' we need to know which instructions to dispatch to the Texture Unit and which to the ALU unit. This information will be encapsulated in the flow control instructions.

Each control flow instruction will contain 2 bits of information for each (non-control flow) instruction:

a) ALU or Texture
b) Serialize Execution

(b) would force the thread to stop execution at this point (before the instruction is executed) and wait until all textures have been fetched. Given the allocation of reserved bits, this would mean that the count of an 'Exec' instruction would be limited to about 8 (non-control-flow) instructions. If more than this were needed, a second Exec (with the same conditions) would be issued.

Another function that relies upon 'clauses' is allocation and order of execution. We need to assure that pixels and vertices are exported in the correct order (even if not all execution is ordered) and that space in the output buffers are allocated in order. Additionally data can't be exported until space is allocated. A new control flow instruction:

Alloc <buffer select -- position,parameter, pixel or vertex memory. And the size required>.

would be created to mark where such allocation needs to be done. To assure allocation is done in order, the actual allocation for a given thread can not be performed unless the equivalent allocation for all previous threads is already completed. The implementation would also assure that execution of instruction(s) following the serialization due to the Alloc will occur in order -- at least until the next serialization or change from ALU to Texture. In most cases this will allow the exports to occur without any further synchronization. Only 'final' allocations or position allocations are guaranteed to be ordered. Because strict ordering is required for pixels, parameters and positions, this implies only a single alloc for these structures. Vertex exports to memory do not require ordering during allocation and so multiple 'allocs' may be done.

6.2.1 *Control flow instructions table*

Here is the revised control flow instruction set.

Note that whenever a field is marked as RESERVED, it is assumed that all the bits of the field are cleared (0).

NOP		
47 … 44	43	42 … 0
0000	Addressing	RESERVED

This is a regular NOP.

Execute						
47 … 44	43	40 … 34	33 … 28	27 …16	15…12	11 … 0
0001	Addressing	RESERVED	Vertex Cache	Instructions type + serialize (6 instructions)	Count	Exec Address

Execute_End						
47 … 44	43	40 … 34	33 … 28	27 …16	15…12	11 … 0
0010	Addressing	RESERVED	Vertex Cache	Instructions type + serialize (6 instructions)	Count	Exec Address

Execute up to 6 instructions at the specified address in the instruction memory. The Instruction type field tells the sequencer the type of the instruction (LSB) (1 = Texture, 0 = ALU and whether to serialize or not the execution (MSB) (1 = Serialize, 0 = Non-Serialized). If the corresponding VC bit is set then VC is used instead of TP/ALU. If Execute_End this is the last execution block of the shader program.

Vertex Cache	Serialize	Instruction Type (Resource)	
0	0	0	: ALU instruction, not yielding
0	0	1	: Texture instruction, not yielding
0	1	0	: ALU instruction, yielding
0	1	1	: Texture instruction, yielding
1	0	0	: Vertex cache instruction, not yielding
1	0	1	: Vertex cache instruction, not yielding
1	1	0	: Vertex cache instruction, yielding
1	1	1	: Vertex cache instruction, yielding

Conditional_Execute							
47 … 44	43	42	41 … 34	33…28	27…16	15 …12	11 … 0
0011	Addressing	Condition	Boolean address	Vertex Cache	Instructions type + serialize (6 instructions)	Count	Exec Address

Conditional_Execute_End							
47 … 44	43	42	41 … 34	33…28	27…16	15 …12	11 … 0
0100	Addressing	Condition	Boolean address	Vertex Cache	Instructions type + serialize (6 instructions)	Count	Exec Address

If the specified Boolean (8 bits can address 256 Booleans) meets the specified condition then execute the specified instructions (up to 9 instructions). If the condition is not met, we go on to the next control flow instruction. If Conditional_Execute_End and the condition is met, this is the last execution block of the shader program.

Conditional_Execute_Predicates								
47 … 44	43	42	41 … 36	35 … 34	33…28	27…16	15…12	11 … 0
0101	Addressing	Condition	RESERVED	Predicate vector	Vertex Cache	Instructions type + serialize (6 instructions)	Count	Exec Address

Conditional_Execute_Predicates_End								
47 … 44	43	42	41 … 36	35 … 34	33…28	27…16	15…12	11 … 0
0110	Addressing	Condition	RESERVED	Predicate vector	Vertex Cache	Instructions type + serialize (6 instructions)	Count	Exec Address

Check the AND/OR of all current predicate bits. If AND/OR matches the condition execute the specified number of instructions. We need to AND/OR this with the kill mask in order not to consider the pixels that aren't valid. If the condition is not met, we go on to the next control flow instruction. If Conditional_Execute_Predicates_End and the condition is met, this is the last execution block of the shader program.

Conditional_Execute_Predicates_No_Stall								
47 … 44	43	42	41 … 36	35 … 34	33…28	27…16	15…12	11 … 0
1101	Addressing	Condition	RESERVED	Predicate vector	Vertex Cache	Instructions type + serialize (6 instructions)	Count	Exec Address

Conditional_Execute_Predicates_No_Stall_End								
47 … 44	43	42	41 … 36	35 … 34	33…28	27…16	15…12	11 … 0
1110	Addressing	Condition	RESERVED	Predicate vector	Vertex Cache	Instructions type + serialize (6 instructions)	Count	Exec Address

Same as Conditionnal_Execute_Predicates but the SQ is not going to wait for the predicate vector to be updated. You can only set this in the compiler if you know that the predicate set is only a refinement of the current one (like a nested if) because the optimization would still work.

Loop_Start					
47 … 44	43	42 … 21	20 … 16	15…13	12 … 0
0111	Addressing	RESERVED	loop ID	RESERVED	Jump address

Loop Start. Compares the loop iterator with the end value. If loop condition not met jump to the address. Forward jump only. Also computes the index value. The loop id must match between the start to end, and also indicates which control flow constants should be used with the loop.

Loop_End									
47 …44	43	42	41… 36	35…34	33… 22	21	20 … 16	15…13	12 … 0
1000	Addressing	Cond	RESERVED	Predicate Vector	RESERVED	Pred break	loop ID	RESERVED	start address

Loop end. Increments the counter by one, compares the loop count with the end value. If loop condition met, continue, else, jump BACK to the start of the loop. If predicate break != 0, then compares predicate vector n (specified by predicate Vector) to condition. If all bits meet condition then break the loop.

The way this is described does not prevent nested loops, and the inclusion of the loop id make this easy to do.

Conditionnal_Call						
47 … 44	43	42	41 … 34	33 … 14	13	12 … 0
1001	Addressing	Condition	Boolean address	RESERVED	Force Call	Jump address

If the condition is met, jumps to the specified address and pushes the control flow program counter on the stack. If force call is set the condition is ignored and the call is made always.

Return	
47 … 44	43	42 … 0
1010	Addressing	RESERVED

Pops the topmost address from the stack and jumps to that address. If nothing is on the stack, the program will just continue to the next instruction.

Conditionnal_Jump							
47 … 44	43	42	41… 34	33	32 … 14	13	12 … 0
1011	Addressing	Condition	Boolean address	FW only	RESERVED	Force Jump	Jump address

If force jump is set the condition is ignored and the jump is made always. If FW only is set then only forward jumps are allowed.

Allocate					
47 … 44	43	42…41	40	39 … 3	2…0
1100	Debug	Buffer Select	No Serial	RESERVED	Size

Buffer Select takes a value of the following:
01 – position export (ordered export)
10 – parameter cache or pixel export (ordered export)
11 – pass thru (out of order exports).

Size field is only used to reserve space in the export buffer for pass thru exports. Valid values are 1 (1 line) thru 5 (5 lines). It should be determined by the compiler/assembler by taking max index used +1.

If debug is set this is a debug alloc (ignore if debug DB_ON register is set to off).

By default the serial bit is set on an alloc. If the No Serial bit is asserted then the serial bit won't be set in the SQ.

6.2.2 *Alloc Statements*

Alloc statements are control flow instructions that allocate resources that are required for executable export instructions. An alloc statement can be either a normal yield point, or a partial yield. At a partial yield - hardware releases the gpu so all state (mova, grad etc) is lost but the thread can resume before all pending fetches have completed.

There are three types of allocs:

alloc-position - proceeds the export of position from a vertex shader. A vertex shader must include one alloc of position. A position alloc cannot be used in a pixel shader. all exports for position must execute between the alloc-position and the next yield point or resource change. There is a small performance advantage to placing the alloc-position near the top of the vertex shader. However we don't think this is worth adding an
extra instruction or register to the shader.

alloc-interp/color - proceeds interpolator exports in a vertex shader or the color exports in a pixel shader. There can be only one alloc interp/color per shader. The color alloc in a pixel shader must be after any alloc-mem-exports. The actual exports can occur anywhere between the alloc-interp/color and the end of the program. There is a small performance advantage to placing the alloc-interp/color near the bottom of the shader. However we don't think this is worth adding an extra instruction or register to the shader. There is a big performance advantage of having no fetches of any kind after the alloc-interp/color.

alloc-mem-export - proceeds any memory-address, memory-data exports. There can be multiple alloc-mem-export statements in either kind of shader. All exports for mem-exports must execute between the corresponding alloc-mem-export and the next yield point or resource change.

6.3 Implementation

The envisioned implementation has a buffer that maintains the state of each thread. A thread lives in a given location in the buffer during its entire life, but the buffer has FIFO qualities in that threads leave in the order that they enter. Actually two buffers are maintained -- one for Vertices and one for Pixels. The intended implementation would allow for:

16 entries for vertices
48 entries for pixels.

From each buffer, arbitration logic attempts to select 1 thread for the texture unit and 2 (interleaved) thread for the ALU unit. Once a thread is selected it is read out of the buffer, marked as invalid, and submitted to appropriate execution unit. It is returned to the buffer (at the same place) with its status updated once all possible sequential

instructions have been executed. A switch from ALU to TEX or visa-versa or a Serialize_Execution modifier forces the thread to be returned to the buffer.

Each entry in the buffer will be stored across two physical pieces of memory - most bits will be stored in a 1 read port device. Only bits needed for thread arbitration will be stored in a highly multi-ported structure. The bits kept in the 1 read port device will be termed 'state'. The bits kept in the multi-read ported device will be termed 'status'.

'State Bits' needed include:

1. Control Flow Instruction Pointer (13 bits),
2. Execution Count Marker 4 bits),
3. Loop Iterators (4x9 bits),
4. Loop Counters (4x9 bits),
5. Call return pointers (4x13 bits),
6. Predicate Bits (64 bits),
7. Export ID (4 bits),
8. Parameter Cache base Ptr (7 bits),
9. GPR Base Ptr (8 bits),
10. Context Ptr (3 bits).
11. LOD corrections (6x16 bits)
12. Valid bits (64 bits)
13. RT (1 bit) Signifies that this thread is a Real Time thread. This bit must be sent to the Constant store state machine when reading it.

Absent from this list are 'Index' pointers. These are costly enough that I'm presuming that they are instead stored in the GPRs. The first seven fields above (Control Flow Ptr, Execution Count, Loop Counts, call return ptrs, Predicate bits, PC base ptr and export ID) are updated every time the thread is returned to the buffer based on how much progress has been mode on thread execution. GPR Base Ptr, Context Ptr and LOD corrections are unchanged throughout execution of the thread.

'Status Bits' needed include:

- Valid Thread
- ALU engine needed
- Texture engine needed
- VC engine needed
- Texture Reads are outstanding
- VC Reads are outstanding
- Alu bank (0/1)
- Waiting on Texture Read to Complete
- Allocation Wait (2 bits)
- 00 – No allocation needed
- 01 – Position export allocation needed (ordered export)
- 10 – Parameter or pixel export needed (ordered export)
- 11 – pass thru (out of order export)
- Allocation Size (4 bits)
- Position Allocated
- Mem/Color Allocated
- First thread of a new context
- Event thread (NULL thread that needs to trickle down the pipe)
- Last (1 bit)
- Pulse SX (1 bit)

All of the above fields from all of the entries go into the arbitration circuitry. The arbitration circuitry will select a winner for both the Texture Engine and for the ALU engine. There are actually two sets of arbitration -- one for pixels and one for vertices. A final selection is then done between the two. But the rest of this implementation summary only considers the 'first' level selection which is similar for both pixels and vertices.

Texture arbitration requires no allocation or ordering so it is purely based on selecting the 'oldest' thread that requires the Texture Engine.

ALU arbitration is a little more complicated. First, only threads where either of Texture_Reads_outstanding or Waiting_on_Texture_Read_to_Complete are '0' are considered. Then if Allocation_Wait is active, these threads are further filtered based on whether space is available. If the allocation is position allocation, then the thread is only considered if all 'older' threads have already done their position allocation (position allocated bits set). If the allocation is parameter or pixel allocation, then the thread is only considered if it is the oldest thread. Also a thread is not considered if it is a parameter or pixel or position allocation, has its First_thread_of_a_new_context bit set and would cause ALU interleaving with another thread performing the same parameter or pixel or position allocation. Finally the 'oldest' of the threads that pass through the above filters is selected. If the thread needed to allocate, then at this time the allocation is done, based on Allocation_Size. If a thread has its “last” bit set, then it is also removed from the buffer, never to return.

If I now redefine 'clauses' to mean 'how many times the thread is removed from the thread buffer for the purpose of exection by either the ALU or Texture engine', then the minimum number of clauses needed is 2 -- one to perform the allocation for exports (execution automatically halts after an 'Alloc' instruction) (but doesn't performs the actual allocation) and one for the actual ALU/export instructions. As the 'Alloc' instruction could be part of a texture clause (presumably the final instruction in such a clause), a thread could still execute in this minimal number of 2 clauses, even if it involved texture fetching.

The Texture_Reads_Outstanding and VC_reads_Outstanding bits tell the SQ that a texture or VC read is outstanding. In this case, if we encounter a serial bit we need to wait until both resources are free (pending = 0) in order to proceed.

6.4 Data dependant predicate instructions

Data dependant conditionals will be supported in the R400. The only way we plan to support those is by supporting three vector/scalar predicate operations of the form:

PRED_SETE_PUSH - similar to SETE except that the result is 'exported' to the sequencer.
PRED_SETNE_PUSH - similar to SETNE except that the result is 'exported' to the sequencer.
PRED_SETGT_PUSH - similar to SETGT except that the result is 'exported' to the sequencer
PRED_SETGTE_PUSH - similar to SETGTE except that the result is 'exported' to the sequencer

For the scalar operations only we will also support the two following instructions:

PRED_SETE
PRED_SETNE
PRED_SETGT
PRED_SET_INV
PRED_SET_POP
PRED_SET_CLR
PRED_SET_RESTORE

Details about actual implementation of these opcodes are in the shader pipe architectural spec.

The export is a single bit - 1 or 0 that is sent using the same data path as the MOVA instruction. The sequencer will maintain 1 set of 64 bits predicate vectors (in fact 2 sets because we interleave two programs but only 1 will be exposed) and use it to control the write masking. This predicate is maintained across clause boundaries.

Then we have two conditional execute bits. The first bit is a conditional execute “on” bit and the second bit tells us if we execute on 1 or 0. For example, the instruction:

P0_ ADD_# R0,R1,R2

Is only going to write the result of the ADD into those GPRs whose predicate bit is 0. Alternatively, P1_ADD_# would only write the results to the GPRs whose predicate bit is set. The use of the P0 or P1 without precharging the sequencer with a PRED instruction is undefined.

6.5 HW Detection of PV,PS

Because of the control program, the compiler cannot detect statically dependant instructions. In the case of non-masked writes and subsequent reads the sequencer will insert uses of PV,PS as needed. This will be done by comparing the read address and the write address of consecutive instructions. For masked writes, the sequencer will insert detect wich channels to read from the GPRs and which ones to read from the PV/PS.

6.6 Register file indexing

Because we can have loops in fetch clause, we need to be able to index into the register file in order to retrieve the data created in a fetch clause loop and use it into an ALU clause. The instruction will include the base address for register indexing and the instruction will contain these controls:

Bit7	Bit 6	
0	0	'absolute register'
0	1	'relative register'
1	0	'previous vector'
1	1	'previous scalar'

In the case of an absolute register we just take the address as is. In the case of a relative register read we take the base address and we add to it the loop_index and this becomes our new address that we give to the shader pipe.

The sequencer is going to keep a loop index computed as such:

Index = Loop_iterator*Loop_step + Loop_start.

We loop until loop_iterator = loop_count. Loop_step is a signed value [-128…127]. The computed index value is a 10 bit counter that is also signed. Its real range is [-256,256]. The tenth bit is only there so that we can provide an out of range value to the “indexing logic” so that it knows when the provided index is out of range and thus can make the necessary arrangements.

6.7 Debugging the Shaders

In order to be able to debug the pixel/vertex shaders efficiently, we provide 2 methods.

6.7.1 *Method 1: Debugging registers*

Current plans are to expose 2 debugging, or error notification, registers:
1. address register where the first error occurred
2. count of the number of errors

The sequencer will detect the following groups of errors:
- count overflow
- constant indexing overflow
- register indexing overflow

Compiler recognizable errors:
- jump errors
  relative jump address > size of the control flow program
- call stack
  call with stack full
  return with stack empty

With all the other errors, program can continue to run, potentially to worst-case limits.

If indexing outside of the constant or the register range, causing an overflow error, the hardware is specified to return the value with an index of 0. This could be exploited to generate error tokens, by reserving and initializing the 0th register (or constant) for errors.

{ISSUE : Interrupt to the driver or not?}

6.7.2 *Method 2: Exporting the values in the GPRs*

1) The sequencer will have a debug active, count register and an address register for this mode.

Under the normal mode execution follows the normal course.

Under the debug mode it is assumed that the program is always exporting n debug vectors and that all other exports to the SX block (but for position) will be turned off (changed into NOPs) by the sequencer (even if they occur before the address stated by the ADDR debug register).

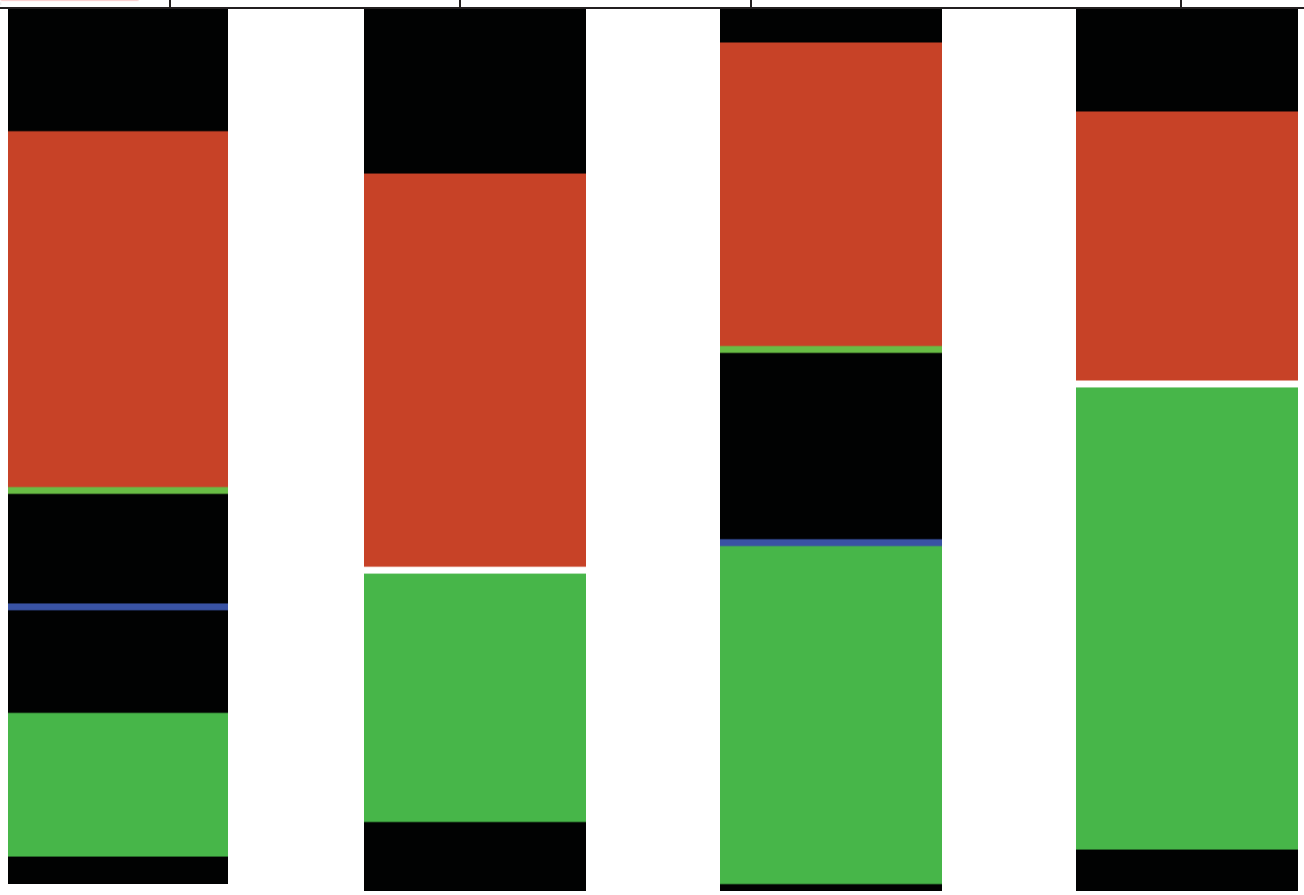
7. Pixel Kill Mask

A vector of 64 bits is kept by the sequencer per group of pixels/vertices. Its purpose is to optimize the texture fetch requests and allow the shader pipe to kill pixels using the following instructions:

MASK_SETE
MASK_SETNE
MASK_SETGT
MASK_SETGTE

8. Register file allocation

The register file allocation for vertices and pixels can either be static or dynamic. In both cases, the register file in managed using two round robins (one for pixels and one for vertices). In the dynamic case the boundary between pixels and vertices is allowed to move, in the static case it is fixed to 128-VERTEX_REG_SIZE for vertices and PIXEL_REG_SIZE for pixels.



Above is an example of how the algorithm works. Vertices come in from top to bottom; pixels come in from bottom to top. Vertices are in orange and pixels in green. The blue line is the tail of the vertices and the green line is the tail of the pixels. Thus anything between the two lines is shared. When pixels meets vertices the line turns white and the boundary is static until both vertices and pixels share the same “unallocated bubble”. Then the boundary is allowed to move again. The numbering of the GPRs starts from the bottom of the picture at index 0 and goes up to the top at index 127.

9. Fetch Arbitration

The fetch arbitration logic chooses one of the n potentially pending fetch clauses to be executed. The choice is made by looking at the Vs and Ps reservation stations and picking the first one ready to execute. Once chosen, the clause state machine will send one 2x2 fetch per clock (or 4 fetches in one clock every 4 clocks) until all the fetch instructions of the clause are sent. This means that there cannot be any dependencies between two fetches of the same clause.

The arbitrator will not wait for the fetches to return prior to selecting another clause for execution. The fetch pipe will be able to handle up to X(?) in flight fetches and thus there can be a fair number of active clauses waiting for their fetch return data.

10. VC Arbitration

The VC arbitration logic chooses one of the n potentially pending VC clauses to be executed. The choice is made by looking at the Vs and Ps reservation stations and picking the first one ready to execute. Once chosen, the clause state machine will send one 2x2 fetch per clock (or 4 fetches in one clock every 4 clocks) until all the fetch instructions of the clause are sent. This means that there cannot be any dependencies between two fetches of the same clause.

The arbitrator will not wait for the fetches to return prior to selecting another clause for execution. The VC pipe will be able to handle up to X(?) in flight VC fetches and thus there can be a fair number of active clauses waiting for their fetch return data.

11. ALU Arbitration

ALU arbitration proceeds in almost the same way than fetch arbitration. The ALU arbitration logic chooses one of the n potentially pending ALU clauses to be executed. The choice is made by looking at the Vs and Ps reservation stations and picking the first one ready to execute. There are two ALU arbiters, one for the even clocks and one for the odd clocks. For example, here is the sequencing of two interleaved ALU clauses (E and O stands for Even and Odd sets of 4 clocks):

Einst0 Oinst0 Einst1 Oinst1 Einst2 Oinst2 Einst0 Oinst3 Einst1 Oinst4 Einst2 Oinst0…
Proceeding this way hides the latency of 8 clocks of the ALUs. Also note that the interleaving also occurs across clause boundaries.

12. Handling Stalls

When the output file is full, the sequencer prevents the ALU arbitration logic from selecting the last clause (this way nothing can exit the shader pipe until there is place in the output file. If the packet is a vertex packet and the position buffer is full (POS_FULL) then the sequencer also prevents a thread from entering an exporting clause. The sequencer will set the OUT_FILE_FULL signal n clocks before the output file is actually full and thus the ALU arbiter will be able read this signal and act accordingly by not preventing exporting clauses to proceed.

12.1 SP stall conditions

12.1.1 *PS Stalls*

None.

12.1.2 *PV Stalls*

None.

13. Content of the reservation station FIFOs

The reservation FIFOs contain the state of the vector of pixels and vertices. We have two sets of those: one for pixels, and one for vertices. They contain 3 bits of Render State 7 bits for the base address of the GPRs, some bits for LOD correction and coverage mask information in order to fetch fetch for only valid pixels, the quad address.

14. The Output File

The output file is where pixels are put before they go to the RBs. The write BW to this store is 256 bits/clock. Just before this output file are staging registers with write BW 512 bits/clock and read BW 256 bits/clock. The staging registers are 4x128 (and there are 16 of those on the whole chip).

15. IJ Format

The IJ information sent by the PA is of this format on a per quad basis:

We have a vector of IJ's (one IJ per pixel at the centroid of the fragment or at the center of the pixel depending on the mode bit). All pixel's parameters are always interpolated at full 20x24 mantissa precision.

$$P0 = A + I(0) * (B - A) + J(0) * (C - A)$$
$$P1 = A + I(1) * (B - A) + J(1) * (C - A)$$
$$P2 = A + I(2) * (B - A) + J(2) * (C - A)$$
$$P3 = A + I(3) * (B - A) + J(3) * (C - A)$$

P0	P1
P2	P3

Multiplies (Full Precision): 8
Subtracts 19x24 (Parameters): 2
Adds: 8

FORMAT OF P's IJ : Mantissa 20 Exp 4 for I + Sign
Mantissa 20 Exp 4 for J + Sign

Total number of bits : 20*8 + 4*8 + 4*2 = 200.

All numbers are kept using the un-normalized floating point convention: if exponent is different than 0 the number is normalized if not, then the number is un-normalized. The maximum range for the IJs (Full precision) is +/- 1024.

15.1 Interpolation of constant attributes

Because of the floating point imprecision, we need to take special provisions if all the interpolated terms are the same or if two of the terms are the same.

16. Staging Registers

In order for the reuse of the vertices to be 14, the sequencer will have to re-order the data sent IN ORDER by the VGT for it to be aligned with the parameter cache memory arrangement. Given the following group of vertices sent by the VGT:

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 || 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 || 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 || 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63

The sequencer will re-arrange them in this fashion:

0 1 2 3 16 17 18 19 32 33 34 35 48 49 50 51 || 4 5 6 7 20 21 22 23 36 37 38 39 52 53 54 55 || 8 9 10 11 24 25 26 27 40 41 42 43 56 57 58 59 || 12 13 14 15 28 29 30 31 44 45 46 47 60 61 62 63

The || markers show the SP divisions. In the event a shader pipe is broken, the SQ is responsible to insert padding to account for the missing pipe. For example, if SP1 is broken, vertices 4 5 6 7 20 21 22 23 36 37 38 39 52 53 54 55 will not be sent by the VGT to the SQ **AND** the SQ is responsible to "jump" over these vertices in order for no valid vertices to be sent to an invalid SP.

The most straightforward, *non-compressed* interface method would be to convert, in the VGT, the data to 32-bit floating point prior to transmission to the VSISRs. In this scenario, the data would be transmitted to (and stored in) the VSISRs in full 32-bit floating point. This method requires three 24-bit fixed-to-float converters in the VGT. Unfortunately, it also requires and additional 3,072 bits of storage across the VSISRs. This interface is illustrated in Figure 9. The area of the fixed-to-float converters and the VSISRs for this method is roughly estimated as 0.759sqmm using the R300 process. The gate count estimate is shown in Figure 8.

Basis for 8-deep Latch Memory (from R300)

8x24-bit 11631 μ^2 60.57813 μ^2 per bit

Area of 96x8-deep Latch Memory 46524 μ^2

Area of 24-bit Fix-to-float Converter 4712 μ^2 per converter

Method 1

Block	Quantity	Area
F2F	3	14136
8x96 Latch	16	744384
		758520 μ^2

Figure 8:Area Estimate for VGT to Shader Interface

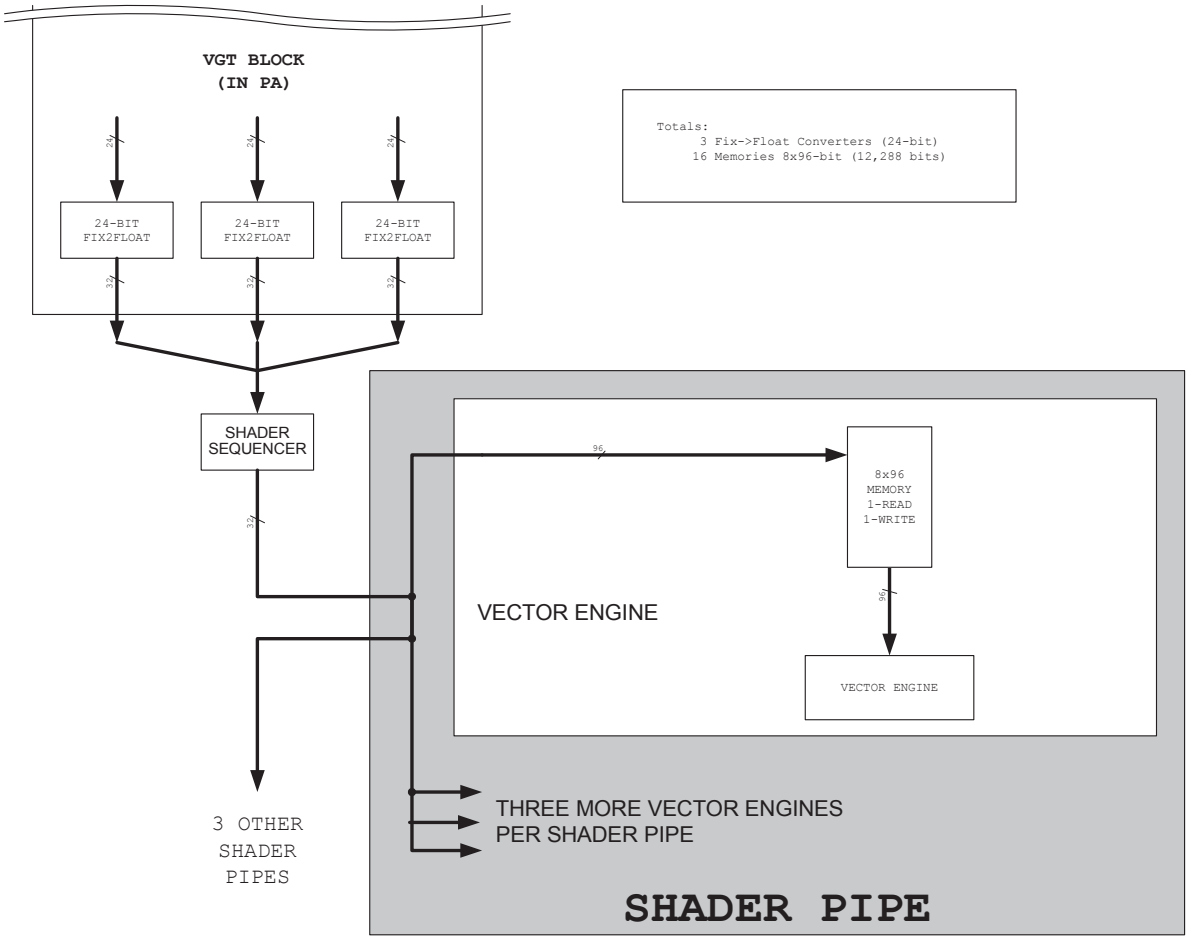


Figure 9:VGT to Shader Interface

17. The parameter cache

The parameter cache is where the vertex shaders export their data. It consists of 16 128x128 memories (1R/1W). The reuse engine will make it so that all vertexes of a given primitive will hit different memories. The allocation

method for these memories is a simple round robin. The parameter cache pointers are mapped in the following way: 4MSBs are the memory number and the 7 LSBs are the address within this memory.

MEMORY NUMBER 4 bits	ADDRESS 7 bits

The PA generates the parameter cache addresses as the positions come from the SQ. All it needs to do is keep a Current_Location pointer (7 bits only) and as the positions comes increment the memory number. When the memory number field wraps around, the PA increments the Current_Location by VS_EXPORT_COUNT (a snooped register from the SQ). As an example, say the memories are all empty to begin with and the vertex shader is exporting 8 parameters per vertex (VS_EXPORT_COUNT = 8). The first position received is going to have the PC address 00000000000 the second one 00010000000, third one 00100000000 and so on up to 11110000000. Then the next position received (the 17th) is going to have the address 00000001000, the 18th 00010001000, the 19th 00100001000 and so on. The Current_location is NEVER reset BUT on chip resets. The only thing to be careful about is that if the SX doesn't send you a full group of positions (<64) then you need to fill the address space so that the next group starts correctly aligned (for example if you receive only 33 positions then you need to add 2*VS_EXPORT_COUNT to Current_Location and reset the memory count to 0 before the next vector begins).

17.1 Export restrictions

17.1.1 *Pixel exports:*

Pixels can export 1,2,3 or 4 color buffers to the SX(+z). The exports will be done in order. The exports will always be ordered to the SX.

17.1.2 *Vertex exports:*

Position or parameter caches can be exported in any order in the shader program. It is always better to export posistion as soon as possible. Position has to be exported in a single export block (no texture instructions can be placed between the exports). Parameter cache exports can be done in any order with texture instructions interleaved. The exports will always be allocated in order to the SX.

17.1.3 *Pass thru exports:*

Pass thru exports have to be done in groups of the form:

```
Alloc 1 thru 5 (max export offset + 1, for example if using EM4 alloc size 5)
Execute ALU(ADDR) ALU(DATA) ALU(DATA) ALU(DATA)…
```

When exporting to more than EM0, one MUST write to EM4 also (the write may be predicated if you don't need the export). This is used to initialize the buffers in the SX.

There cannot be any serialize bits set OR texture Reads between the EA and the last EM.

Memory exports will be surfaced using a macro extension; here is what needs to happen inside the macro:

The macro needs to create a special constant of the form:

Stream ID constant:

.x = Integer that holds BaseAddressInBytes/4 in bits (29:0). Bits 31:30 should be 0b01.
.y = 2**23
.z = Integer that holds register field data. Note that this data must be organized so that it always represents a 'valid' floating point number, with the relevant bits in (23 - 0); One way of doing this would be to take the 23 bits and add 2**23.
.w = max index value + 2**23

Output to EXaddress:

.x = Base of array (in low 30 bits)/4

.y = Index value (in low 23 bits)
.z = Register Field data (in low 23 bits)
.w = Max Index value (in low 23 bits)

Also Assume that C0:

.x = 0.0
.y = 1.0

The Macro expansion would be as follows:

```
MULADD    EA = Rindex.xxxx,C0.xyxx,CstreamID;
MOV       EMx (x = 0 thru 4) = Rdata;
```

The SX will check for invalid writes and **mask out the data** so it won't be written to memory. Invalid writes are:

1) Index value >= Max Index value
2) bit 31 != 0 (negative index)
3) bits [30:23] != 23 + IEEE_EXP_BIAS (127) (meaning the index was too big to be represented using 23 bits)

They cannot have texture instructions interleaved in the export block. These exports **are not guaranteed to be ordered**.

Also, when doing a pass thru export**,** the shader must still do either a position and PC export (if Vertex) or a color export (if Pixel). The pass thru export can occur anywhere in any shader program and thus can be used to debug. There can be any number of pass thru export blocks throughout the pixel or vertex shader or both.

17.2 Arbitration restrictions

Here are the Sequencer arbitration restrictions:

1) Cannot execute a serialized thread if the corresponding texture pending bit and VC pending is set
2) Cannot allocate position if any older thread has not allocated position
3) Cannot execute a texture clause if texture reads are pending
4) Cannot execute a VC clause if VC reads are pending
5) Cannot execute last if texture pending (even if not serial)
6) Cannot allocate if not last for color exports.
7) Cannot allocate if not last for PC exports.

18. Export Types

The export type (or the location where the data should be put) is specified using the destination address field in the ALU instruction. Here is a list of all possible export modes:

18.1 Vertex Shading

0:15 - 16 parameter cache
16:31 - Empty (Reserved?)
32 - Export Address
33:37 - 5 vertex exports to the frame buffer and index
38:46 - Empty
47 - Debug Address
48:52 - 5 debug export (interpret as normal memory export)
53:59 - Empty
60 - export addressing mode
61 - Empty
62 - position
63 - sprite size export that goes with position export

(X= point size, Y= edge flag is bit 0, Z= VtxKill is bitwise OR of bits 30:0. Any bit other than sign means VtxKill.)

18.2 Pixel Shading

0 - Color for buffer 0 (primary)
1 - Color for buffer 1
2 - Color for buffer 2
3 - Color for buffer 3
4:15 - Empty
16 - Buffer 0 Color/Fog (primary)
17 - Buffer 1 Color/Fog
18 - Buffer 2 Color/Fog
19 - Buffer 3 Color/Fog
20:31 - Empty
32 - Export Address
33:37 - 5 exports for multipass pixel shaders.
38:46 - Empty
47 - Debug Address
48:52 - 5 debug exports (interpret as normal memory export)
60 - export addressing mode
61 - Z for primary buffer (Z exported to 'alpha' component)
62:63 - Empty

19. Special Interpolation modes

19.1 Real time commands

We are unable to use the parameter memory since there is no way for a command stream to write into it. Instead we need to add three 4x128 memories (one for each of three vertices x 4 interpolants). These will be mapped onto the register bus and written by type 0 packets, and output to the the parameter busses (the sequencer and/or PA need to be able to address the reatime parameter memory as well as the regular parameter store. This mode is triggered by the primitive type: REAL TIME. The actual memories are in the in the SX blocks. The parameter data memories are hooked on the RBBM bus and are loaded by the CP using register mapped memory.

19.2 Sprites/ XY screen coordinates/ FB information

XY screen coordinates may be needed in the shader program. This functionality is controlled by the param_gen register (in SQ) in conjunction with the SND_XY register (in SC) and the param_gen_pos. Also it is possible to send the faceness information (for OGL front/back special operations) to the shader using the same control register. Here is a list of all the modes and how they interact together:

The Data is going to be written in the register specified by the param_gen_pos register.

Param_Gen disable, snd_xy disable = No modification
Param_Gen disable, snd_xy enable = No modification
Param_Gen enable, snd_xy disable = Sign(faceness)garbage,(Sign Point)garbage,Sign(Line)s, t
Param_Gen enable, snd_xy enable = Sign(faceness)screenX,(Sign Point)screenY,Sign(Line)s, t

In other words,

The generated vector is (X in RED, Y in GREEN, S in BLUE and T in ALPHA):
X,Y,S,T

PGenReg.X = screen X biased 2^23 (assumes pixel center at 0.0), sign bit encodes faceness (0=frontface, 1=backface)
PGenReg.Y = screen Y biased 2^23 (assumes pixel center at 0.0), sign encodes is point primitive (0=not point, 1=is point)
PGenReg.Z = parametric S coordinate [0..1], sign encodes is line primitive (0=not line, 1=is line)

PGenReg.W = parametric T coordinate [0..1]

Constant
C0.X = 2^23 (debias for D3D)
C0.Y= 2^23 - 0.5 (debias for OGL which has pixel centers at 0.5)

To generate useable XY:
For D3D:
ADD ScreenXYReg.xy__ = abs(PGenReg), -C0.xxxx
For OGL
ADD ScreenXYReg.xy__ = abs(PGenReg), -C0.yyyy
Note abs has to be done on PGenReg

To access faceness.
Must ALWAYS use (or pos/neg test against) PGenReg.X.
< 0.0 is backface
>= 0.0 is frontface

To access parametric ST.
Same as before simply take abs before access.
realS = abs(PGenReg.Z)
realT = abs(PGenReg.W)

To access primitive type
+/-ZERO cannot be differentiated in shader pipe so a RECIP_CLAMPED instruction must be done first before testing isLine.
isPoint = PGenReg.Y (if <0.0 then point primitive)
isLine = RECIP_CLAMPED PGenReg.Z (if <0.0 then line primitive)
if ((isPoint>=0.0) && (isLine>=0.0)) then triangle primitive

19.3 Auto generated counters

In the cases we are dealing with multipass shaders, the sequencer is going to generate a vector count to be able to both use this count to write the 1st pass data to memory and then use the count to retrieve the data on the 2nd pass. The count is always generated in the same way but it is passed to the shader in a slightly different way depending on the shader type (pixel or vertex). This is toggled on and off using the GEN_INDEX_PIX/VTX register. The sequencer is going to keep two counters, one for pixels and one for vertices. Every time a full vector of vertices or pixels is written to the GPRs the counter is incremented. Every time a RST_PIX_COUNT or RST_VTX_COUNT events are received, the corresponding counter is reset. While there is only one count broadcast to the GPRs, the LSB are hardwired to specific values making the index different for all elements in the vector. Since the count must be different for all pixels/vertices and the 4 LSBs (16 positions) are hardwired to the corresponding shader unit the SQ has two choices:

1) Maintain a 17 bit counter that counts the vectors of 64. In this case the phase must be appended to the count before the count is broadcast to the SPs:

Counter (17 bits)	Phase (2 bits)	Hardwired (4 bits)

2) Maintain a 21 bits counter that counts sub-vectors of 16. In this case only the counter is sent to the Sps:

Counter (19 bits)	Hardwired (4 bits)

19.3.1 *Vertex shaders*

In the case of vertex shaders, if GEN_INDEX_VTX is set, the data will be put into the x field of the third register (it means that the compiler must allocate 3 GPRs in all multipass vertex shader modes).

19.3.2 *Pixel shaders*

In the case of pixel shaders, if GEN_INDEX_PIX is set, the data will be put in the x field of the param_gen_pos+1 register.

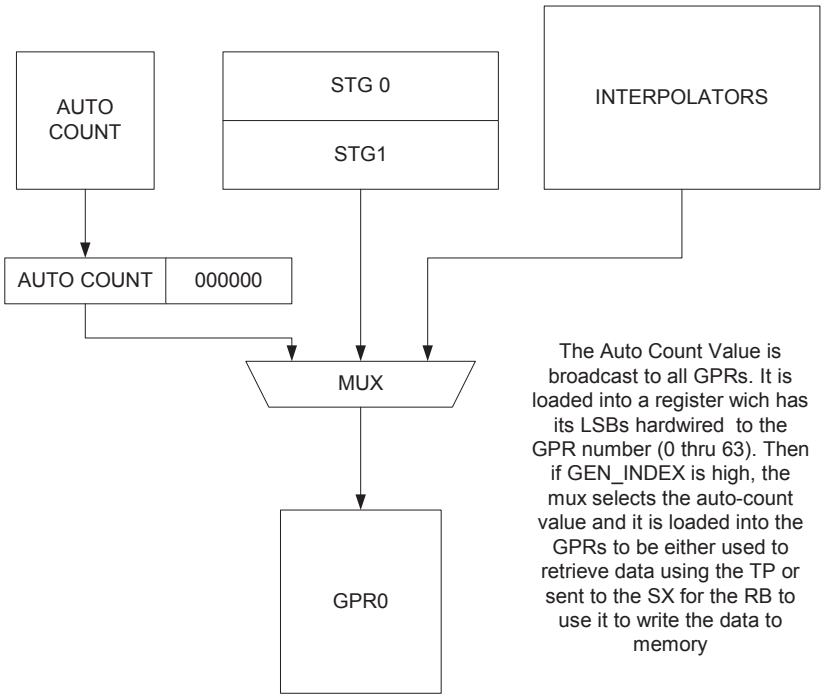


Figure 10: GPR input mux Control

20. State management

Every clock, the sequencer will report to the CP the oldest states still in the pipe. These are the states of the programs as they enter the last ALU clause.

20.1 Parameter cache synchronization

In order for the sequencer not to begin a group of pixels before the associated group of vertices has finished, the sequencer will keep a 6 bit count per state (for a total of 8 counters). These counters are initialized to 0 and every time a vertex shader exports its data TO THE PARAMETER CACHE, the corresponding pointer is incremented. When the SC sends a new vector of pixels with the SC_SQ_new_vector bit asserted, the sequencer will first check if the count is greater than 0 before accepting the transmission (it will in fact accept the transmission but then lower its ready to receive). Then the sequencer waits for the count to go to one and decrements it. The sequencer can then issue the group of pixels to the interpolators. Every time the state changes, the new state counter is initialized to 0.

21. XY Address imports

The SC will be able to send the XY addresses to the GPRs. It does so by interleaving the writes of the IJs (to the IJ buffer) with XY writes (to the XY buffer). Then when writing the data to the GPRs, the sequencer is going to interpolate the IJ data or pass the XY data thru a Fix→float converter and expander and write the converted values to the GPRs. The Xys are currently SCREEN SPACE COORDINATES. The values in the XY buffers will wrap. See section 19.2 for details on how to control the interpolation in this mode.

21.1 Vertex indexes imports

In order to import vertex indexes, we have 16 8x96 staging registers. These are loaded one line at a time by the VGT block (96 bits). They are loaded in floating point format and can be transferred in 4 or 8 clocks to the GPRs.

22. Registers

Please see the auto-generated web pages for register definitions.

23. Interfaces

23.1 External Interfaces

Whenever an x is used, it means that the bus is broadcast to all units of the same name. For example, if a bus is named SQ→SPx it means that SQ is going to broadcast the same information to all SP instances.

23.2 SC to SP Interfaces

23.2.1 *SC_SP#*

There is one of these interfaces at front of each of the SP (buffer to stage pixel interpolators). This interface transmits the I,J data for pixel interpolation. For the entire system, two quads per clock are transferred to the 4 SPs, so each of these 4 interfaces transmits one half of a quad per clock. The interface below describes a half of a quad worth of data.
The actual data which is transferred per quad is

Ref Pix I => S4.20 Floating Point I value *4
Ref Pix J => S4.20 Floating Point J value *4

This equates to a total of 200 bits which transferred over 2 clocks
and therefor needs an interface 100 bits wide

Additionally, X,Y data (12-bit unsigned fixed) is conditionally sent across this data bus over the same wires in an additional clock. The X,Y data is sent on the lower 24 bits of the data bus with faceness in the msb.
Transfers across these interfaces are synchronized with the SC_SQ IJ Control Bus transfers.

The data transfer across each of these busses is controlled by a IJ_BUF_INUSE_COUNT in the SC. Each time the SC has sent a pixel vector's worth of data to the SPs, he will increment the IJ_BUF_INUSE_COUNT count. Prior to sending the next pixel vectors data, he will check to make sure the count is less than MAX_BUFER_MINUS_2, if not the SC will stall until the SQ returns a pipelined pulse to decrement the count when he has scheduled a buffer free. Note: We could/may optimize for the case of only sending only IJ to use all the buffers to pre-load more. Currently it is planned for the SP to hold 2 double buffers of I,J data and two buffers of X,Y data, so if either X,Y or Centers and Centroids are on, then the SC can send two Buffers.

In at least the initial version, the SC shall send 16 quads per pixel vector even if the vector is not full. This will increment buffer write address pointers correctly all the time. (We may revisit this for both the SX,SP,SQ and add a EndOfVector signal on all interfaces to quit early. We opted for the simple mode first with a belief that only the end of packet and multiple new vector signals should cause a partial vector and that this would not really be significant performance hit.)

Name	Bits	Description
SC_SP#_data	100	IJ information sent over 2 clocks (or X,Y in 24 LSBs with faceness in upper bit) **Type 0 or 1**, First clock I, second clk J Field ULC URC LLC LRC Bits [63:39] [38:26] [25:13] [12:0] Format SE4M20 SE4M20 SE4M20 SE4M20 **Type 2** Field Face X Y Bits [24] [23:12] [11:0] Format Bit Unsigned Unsigned
SC_SP#_valid	1	Valid
SC_SP#_last_quad_data	1	This bit will be set on the last transfer of data per quad.

SC_SP#_type	2	0 -> Indicates centroids 1 -> Indicates centers 2 -> Indicates X,Y Data and faceness on data bus The SC shall look at state data to determine how many types to send for the interpolation process.

The # is included for clarity in the spec and will be replaced with a prefix of u#_ in the verilog module statement for the SC and the SP block will have neither because the instantiation will insert the prefix.

23.2.2 SC_SQ

This is the control information sent to the sequencer in order to synchronize and control the interpolation and/or loading data into the GPRs needed to execute a shader program on the sent pixels. This data will be sent over two clocks per transfer with 1 to 16 transfers. Therefore the bus (approx 108 bits) could be folded in half to approx 54 bits.

Name	Bits	Description
SC_SQ_data	46	Control Data sent to the SQ 1 clk transfers Event – valid data consist of event_id and state_id. Instruct SQ to post an event vector to send state id and event_id through request fifo and onto the reservation stations making sure state id and/or event_id gets back to the CP. Events only follow end of packets so no pixel vectors will be in progress. Empty Quad Mask – Transfer Control data consisting of pc_dealloc or new_vector. Receipt of this is to transfer pc_dealloc or new_vector without any valid quad data. New vector will always be posted to request fifo and pc_dealloc will be attached to any pixel vector outstanding or posted in request fifo if no valid quad outstanding. 2 clk transfers Quad Data Valid – Sending quad data with or without new_vector or pc_dealloc. New vector will be posted to request fifo with or without a pixel vector and pc_dealloc will be posted with a pixel vector unless none is in progress. In this case the pc_dealloc will be posted in the request queue. Filler quads will be transferred with The Quad mask set but the pixel corresponding pixel mask set to zero.
SC_SQ_valid	1	SC sending valid data, 2nd clk could be all zeroes

SC_SQ_data – first clock and second clock transfers are shown in the table below.

Name	BitField	Bits	Description

1st Clock Transfer			
SC_SQ_event	0	1	This transfer is a 1 clock event vector Force quad_mask = new_vector=pc_dealloc=0
SC_SQ_event_id	[5:1]	4	This field identifies the event 0 => denotes an End Of State Event 1 => TBD
SC_SQ_state_id	[8:6]	3	State/constant pointer (6*3+3)
SC_SQ_pc_dealloc	[11:9]	3	Deallocation token for the Parameter Cache
SC_SQ_new_vector	12	1	The SQ must wait for Vertex shader done count > 0 and after dispatching the Pixel Vector the SQ will decrement the count.
SC_SQ_quad_mask	[16:13]	4	Quad Write mask left to right SP0 => SP3
SC_SQ_end_of_prim	17	1	End Of the primitive
SC_SQ_pix_mask	[33:18]	16	Valid bits for all pixels SP0=>SP3 (UL,UR,LL,LR)
SC_SQ_provok_vtx	[35:34]	2	Provoking vertex for flat shading
SC_SQ_lod_correct_0	[44:36]	9	LOD correction for quad 0 (SP0) (9 bits per quad)
SC_SQ_lod_correct_1	[53:45]	9	LOD correction for quad 1 (SP1) (9 bits per quad)
2nd Clock Transfer			
SC_SQ_lod_correct_2	[8:0]	9	LOD correction for quad 2 (SP2) (9 bits per quad)
SC_SQ_lod_correct_3	[17:9]	9	LOD correction for quad 3 (SP3) (9 bits per quad)
SC_SQ_pc_ptr0	[28:18]	11	Parameter Cache pointer for vertex 0
SC_SQ_pc_ptr1	[39:29]	11	Parameter Cache pointer for vertex 1
SC_SQ_pc_ptr2	[50:40]	11	Parameter Cache pointer for vertex 2
SC_SQ_prim_type	[53:51]	3	Stippled line and Real time command need to load tex cords from alternate buffer 000: Sprite (point) 001: Line 010: Tri_rect 100: Realtime Sprite (point) 101: Realtime Line 110: Realtime Tri_rect

Name	Bits	Description
SQ_SC_free_buff	1	Pipelined bit that instructs SC to decrement count of buffers in use.
SQ_SC_dec_cntr_cnt	1	Pipelined bit that instructs SC to decrement count of new vector and/or event sent to prevent SC from overflowing SQ interpolator/Reservation request fifo.

The scan converter will submit a partial vector whenever:

1.) He gets a primitive marked with an end of packet signal.

2.) A current pixel vector is being assembled with at least one or more valid quads and the vector has been marked for deallocate when a primitive marked new_vector arrives. The Scan Converter will submit a partial vector (up to 16quads with zero pixel mask to fill out the vector) prior to submitting the new_vector marker\primitive.

(This will prevent a hang which can be demonstrated when all primitives in a packet three vectors are culled except for a one quad primitive that gets marked pc_dealloc (vertices maximum size). In this case two new_vectors are submitted and processed, but then one valid quad with the pc_dealloc creates a vector and then the new would wait for another vertex vector to be processed, but the one being waited for could never export until the pc_dealloc signal made it through and thus the hang.)

23.2.3 SQ to SX(SP): Interpolator bus

Name	Direction	Bits	Description
SQ_SXx_interp_cyl_wrap	SQ→SXx	4	Which channel needs to be cylindrical wrapped
SQ_SXx_wrap_count	SQ->SXx	2	Cylindrical wrap count
SQ_SPx_auto_count	SQ->SPx	19	Auto generated count for VTx and Pixels
SQ_SPx_interp_param_gen	SQ→SPx	1	Generate Parameter
SQ_SPx_interp_prim_type	SQ→SPx	2	Bits [1:0] of primitive type sent by SC
SQ_SPx_interp_buff_swap	SQ→SPx	1	Swap IJ buffers
SQ_SPx_interp_IJ_line	SQ→SPx	2	IJ line number
SQ_SPx_interp_mode	SQ→SPx	1	Center/Centroid sampling
SQ_SXx_pc_ptr0	SQ→SXx	11	Parameter Cache Pointer
SQ_SXx_pc_ptr1	SQ→SXx	11	Parameter Cache Pointer
SQ_SXx_pc_ptr2	SQ→SXx	11	Parameter Cache Pointer
SQ_SXx_rt_sel	SQ→SXx	1	Selects between RT and Normal data (Bit 2 of prim type)
SQ_SX0_pc_wr_en	SQ→SX0	8	Write enable for the PC memories
SQ_SX1_pc_wr_en	SQ→SX1	8	Write enable for the PC memories
SQ_SXx_pc_wr_addr	SQ→SXx	7	Write address for the PCs
SQ_SXx_pc_channel_mask	SQ→SXx	4	Channel mask
SQ_SXx_pc_ptr_valid	SQ→SXx	1	Read pointers are valid.
SQ_SPx_interp_valid	SQ→SPx	1	Interpolation control valid

23.2.4 SQ to SP: Staging Register Data

This is a broadcast bus that sends the VSISR information to the staging registers of the shader pipes.

Name	Direction	Bits	Description
SQ_SPx_vsr_data	SQ→SPx	96	Pointers of indexes or HOS surface information
SQ_SPx_vsr_wrt_addr	SQ→SPx	3	Staging register write address
SQ_SPx_vsr_rd_addr	SQ→SPx	3	Staging register read address
SQ_SP0_ vsr_valid	SQ→SP0	1	Data is valid
SQ_SP1_ vsr_ valid	SQ→SP1	1	Data is valid
SQ_SP2_ vsr_ valid	SQ→SP2	1	Data is valid
SQ_SP3_ vsr_ valid	SQ→SP3	1	Data is valid
SQ_SPx_vsr_read	SQ→SPx	1	Increment the read pointers

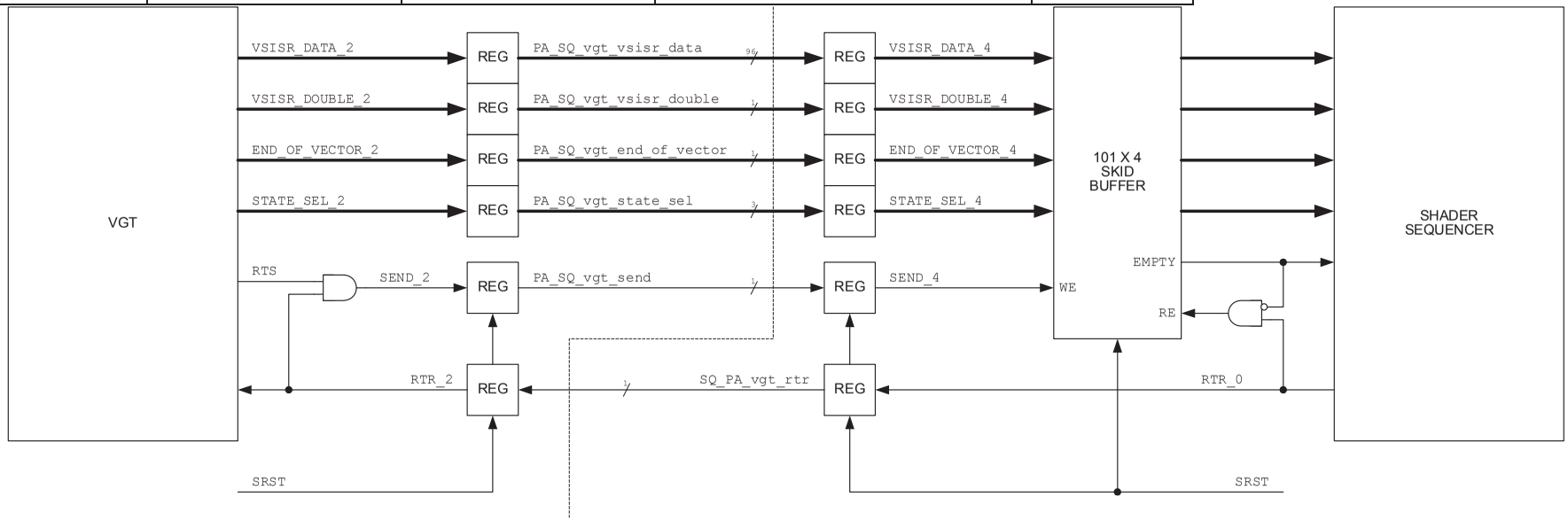
23.2.5 VGT to SQ : Vertex interface

23.2.5.1 Interface Signal Table

The area difference between the two methods is not sufficient to warrant complicating the interface or the state requirements of the VSISRs. **Therefore, the POR for this interface is that the VGT will transmit the data to the VSISRs (via the Shader Sequencer) in full, 32-bit floating-point format.** The VGT can transmit up to six 32-bit floating-point values to each VSISR where four or more values require two transmission clocks. The data bus is 96 bits wide. In the case where an event is sent the 5 LSBs of VGT_SQ_vsisr_data contain the eventID.

Name	Bits	Description
VGT_SQ_vsisr_data	96	Pointers of indexes or HOS surface information
VGT_SQ_event	1	VGT is sending an event
VGT_SQ_vsisr_continued	1	0: Normal 96 bits per vert 1: double 192 bits per vert
VGT_SQ_end_of_vtx_vect	1	Indicates the last VSISR data set for the current process vector (for double vector data, "end_of_vector" is set on the first vector)
VGT_SQ_indx_valid	1	Vsisr data is valid
VGT_SQ_state	3	Render State (6*3+3 for constants). This signal is guaranteed to be correct when “VGT_SQ_vgt_end_of_vector” is high.
VGT_SQ_send	1	Data on the VGT_SQ is valid receive (see write-up for standard R400 SEND/RTR interface handshaking)
SQ_VGT_rtr	1	Ready to receive (see write-up for standard R400 SEND/RTR interface handshaking)

23.2.5.2 Interface Diagrams



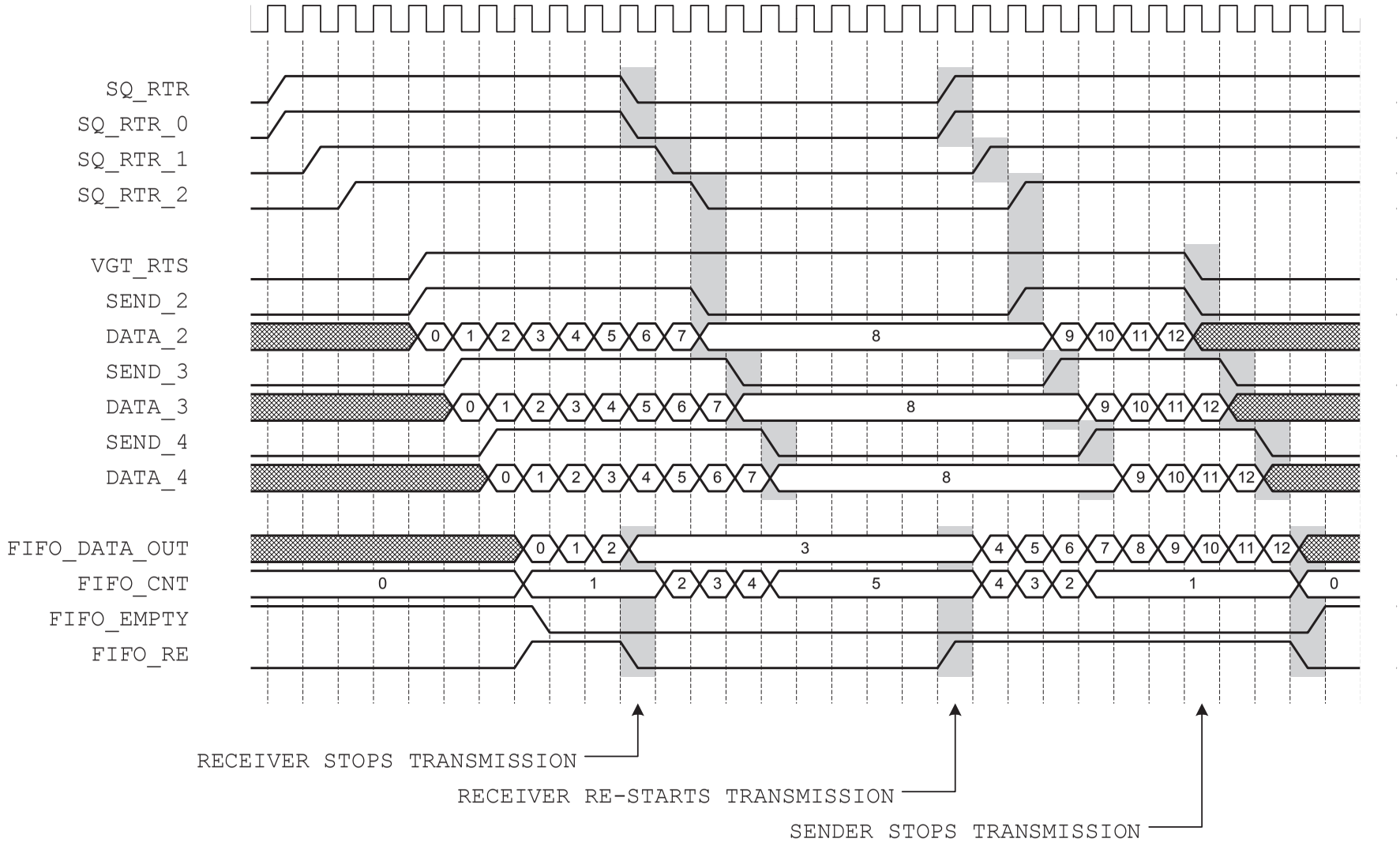


Figure 1. Detailed Logical Diagram for PA_SQ_vgt Interface.

23.2.6 SQ to SX: Control bus

Name	Direction	Bits	Description
SQ_SXx_exp_type	SQ→SXx	2	00: Pixel without z (1 to 4 buffers) 01: Pixel with z (1 to 4 buffers) 10: Position (1 or 2 results) 11: Pass thru (1 to 5 results aligned)
SQ_SXx_exp_number	SQ→SXx	2	Number of locations needed in the export buffer (encoding depends on the type see bellow).
SQ_SXx_exp_alu_id	SQ→SXx	4	ALU ID. Revolving ID 0 thru 15. Memory exports have to increment this count by 4 or 8 depending on the size requested. Other type of exports increment the ID by 1.
SQ_SXx_exp_valid	SQ→SXx	1	Valid bit
SQ_SXx_exp_state	SQ→SXx	3	State Context
SQ_SXx_free_done	SQ→SXx	1	Pulse that indicates that the previous export is finished **from the point of view of the SP. This does not necessarily mean that the data has been transferred to RB or PA, or that the space in export buffer for that particular vector thread has been freed up.**
SQ_SXx_free_alu_id	SQ→SXx	4	ALU ID that was used at allocate time.

Depending on the type the number of export location changes:

- Type 00 : Pixels without Z
  - 00 = 1 buffer
  - 01 = 2 buffers
  - 10 = 3 buffers
  - 11 = 4 buffer
- Type 01: Pixels with Z
  - 00 = 2 Buffers (color + Z)
  - 01 = 3 buffers (2 color + Z)
  - 10 = 4 buffers (3 color + Z)
  - 11 = 5 buffers (4 color + Z)
- Type 10 : Position export
  - 00 = 1 position
  - 01 = 2 positions
  - 1X = Undefined
- Type 11: Pass Thru
  - 00 = 4 buffers
  - 01 = 8 buffers
  - 10 = Undefined
  - 11 = Undefined

Below the thick black line is the end of transfer packet that tells the SX that a given export is finished. The report packet **will always arrive either before or at the same time than the next export to the same ALU id**.

23.2.7 SX to SQ : Output file control

Name	Direction	Bits	Description
SXx_SQ_pix_free_count0	SXx→SQ	6	How many slots where just freed in the SX for bank0
SXx_SQ_pix_count0_valid	SXx→SQ	1	Free_count0 is valid
SXx_SQ_pix_free_count1	SXx→SQ	6	How many slots where just freed in the SX for bank1
SXx_SQ_pix_count1_valid	SXx→SQ	1	Free_count1 is valid
SXx_SQ_pos_free_count0	SXx→SQ	4	How many slots where just freed in the SX for bank0
SXx_SQ_pos_count0_valid	SXx→SQ	1	Free_count0 is valid
SXx_SQ_pos_free_count1	SXx→SQ	4	How many slots where just freed in the SX for bank1

SXx_SQ_pos_count1_valid	SXx→SQ	1	Free_count1 is valid
SXx_SQ_mem_export_free	SXx→SQ	1	Freed a memory export slot

23.2.8 SQ to TP: Control bus

Once every clock, the fetch unit sends to the sequencer on which RS line it is now working and if the data in the GPRs is ready or not. This way the sequencer can update the fetch valid bits flags for the reservation station. The sequencer also provides the instruction and constants for the fetch to execute and the address in the register file where to write the fetch return data.

Name	Direction	Bits	Description
TPx_SQ_data_rdy	TPx→ SQ	1	Data ready
TPx_SQ_rs_line_num	TPx→ SQ	6	Line number in the Reservation station
TPx_SQ_type	TPx→ SQ	1	Type of data sent (0:PIXEL, 1:VERTEX)
SQ_TPx_send	SQ→TPx	1	Sending valid data
SQ_TPx_const	SQ→TPx	48	Fetch state sent over 4 clocks (192 bits total)
SQ_TPx_instr	SQ→TPx	24	Fetch instruction sent over 4 clocks
SQ_TPx_end_of_group	SQ→TPx	1	Last instruction of the group
SQ_TPx_Type	SQ→TPx	1	Type of data sent (0:PIXEL, 1:VERTEX)
SQ_TPx_gpr_phase	SQ→TPx	2	Write phase signal
SQ_TP0_lod_correct	SQ→TP0	6	LOD correct 3 bits per comp 2 components per quad
SQ_TP0_pix_mask	SQ→TP0	4	Pixel mask 1 bit per pixel
SQ_TP1_lod_correct	SQ→TP1	6	LOD correct 3 bits per comp 2 components per quad
SQ_TP1_pix_mask	SQ→TP1	4	Pixel mask 1 bit per pixel
SQ_TP2_lod_correct	SQ→TP2	6	LOD correct 3 bits per comp 2 components per quad
SQ_TP2_pix_mask	SQ→TP2	4	Pixel mask 1 bit per pixel
SQ_TP3_lod_correct	SQ→TP3	6	LOD correct 3 bits per comp 2 components per quad
SQ_TP3_pix_mask	SQ→TP3	4	Pixel mask 1 bit per pixel
SQ_TPx_rs_line_num	SQ→TPx	6	Line number in the Reservation station
SQ_TPx_write_gpr_index	SQ->TPx	7	Index into Register file for write of returned Fetch Data
SQ_TPx_ctx_id	SQ→TPx	3	The state context ID (needed for multisample resolves)
SQ_TPx_SIMD	SQ->TPx	1	Tells the TP from which SIMD the data is coming from.

23.2.9 SQ to VC: Control bus

Once every clock, the VC unit sends to the sequencer on which RS line it is now working and if the data in the GPRs is ready or not. This way the sequencer can update the fetch valid bits flags for the reservation station. The sequencer also provides the instruction and constants for the fetch to execute and the address in the register file where to write the fetch return data.

Name	Direction	Bits	Description
VCx_SQ_data_rdy	VCx→ SQ	1	Data ready
VCx_SQ_rs_line_num	VCx→ SQ	6	Line number in the Reservation station
VCx_SQ_type	VCx→ SQ	1	Type of data sent (0:PIXEL, 1:VERTEX)
SQ_VCx_send	SQ→VCx	1	Sending valid data
SQ_VCx_const	SQ→VCx	48	Fetch state sent over 4 clocks (192 bits total)
SQ_VCx_instr	SQ→VCx	24	Fetch instruction sent over 4 clocks
SQ_VCx_end_of_group	SQ→VCx	1	Last instruction of the group
SQ_VCx_Type	SQ→VCx	1	Type of data sent (0:PIXEL, 1:VERTEX)
SQ_VCx_gpr_phase	SQ→VCx	2	Write phase signal
SQ_VC0_pix_mask	SQ→VC0	4	Pixel mask 1 bit per pixel
SQ_VC1_pix_mask	SQ→VC1	4	Pixel mask 1 bit per pixel
SQ_VC2_pix_mask	SQ→VC2	4	Pixel mask 1 bit per pixel
SQ_VC3_pix_mask	SQ→VC3	4	Pixel mask 1 bit per pixel
SQ_VCx_rs_line_num	SQ→VCx	6	Line number in the Reservation station

SQ_VCx_write_gpr_index	SQ->VCx	7	Index into Register file for write of returned Fetch Data
SQ_VCx_SIMD	SQ->VCx	1	Tells the VC from which SIMD the data is coming from.

23.2.10 *TP to SQ: Texture stall*

The TP sends this signal to the SQ and the SPs when frees up a buffer.

Name	Direction	Bits	Description
TP_SQ_fetch_dec	TP→ SQ	1	Just freed a slot in the TP.

23.2.11 *VC to SQ: Vertex Cache stall*

The VC sends this signal to the SQ and the SPs when frees up a buffer. There are 2 types of buffers, Mega and Mini and a signal for both.

Name	Direction	Bits	Description
VC_SQ_fetch_dec_mega	VC→ SQ	1	Freed a Mega slot in the VC.
VC_SQ_fetch_dec_mini	VC→ SQ	1	Freed a Mini slot in the VC.

23.2.12 *SQ to SP: GPR and auto counter*

Name	Direction	Bits	Description
SQ_SPx_simd0_gpr_wr_addr	SQ→SPx	7	Write address
SQ_SPx_simd0_gpr_rd_addr	SQ→SPx	7	Read address
SQ_SPx_simd0_gpr_rd_en	SQ→SPx	1	Read Enable
SQ_SP0_simd0_gpr_pspv_wr_en	SQ→SP0 (SP1)	4	Write Enable for the GPRs of SP0-1 for PS and PV
SQ_SP2_simd0_gpr_pspv_wr_en	SQ→SP2 (SP3)	4	Write Enable for the GPRs of SP2-3 for PS and PV
SQ_SP4_simd0_gpr_pspv_wr_en	SQ→SP4 (SP5)	4	Write Enable for the GPRs of SP4-5 for PS and PV
SQ_SP6_simd0_gpr_pspv_wr_en	SQ→SP6 (SP7)	4	Write Enable for the GPRs of SP6-7 for PS and PV
SQ_SP0_simd0_gpr_int_wr_en	SQ→SP0	1	Write Enable for the GPRs of SP0 for Inputs (interp/vtx)
SQ_SP2_simd0_gpr_int_wr_en	SQ→SP2	1	Write Enable for the GPRs of SP1 for Inputs (interp/vtx)
SQ_SP4_simd0_gpr_int_wr_en	SQ→SP4	1	Write Enable for the GPRs of SP2 for Inputs (interp/vtx)
SQ_SP6_simd0_gpr_int_wr_en	SQ→SP6	1	Write Enable for the GPRs of SP3 for Inputs (interp/vtx)
SQ_SPx_gpr_phase	SQ→SPx	2	The phase mux (arbitrates between inputs, ALU SRC reads and writes)
SQ_SPx_simd0_channel_mask	SQ→SPx	4	The channel mask for SIMD0
SQ_SPx_gpr_input_sel	SQ→SPx	2	When the phase mux selects the inputs this tells from which source to read from: Interpolated data, VSR, autogen counter.
SQ_SPx_auto_count	SQ→SPx	21	Auto count generated by the SQ, common for all shader pipes
SQ_SPx_simd0_fetch_swizzle	SQ→SPx	6	Swizzle code for the TP request (2 bits per channel ignore W as it is not used). Bits [1..0] X mode select: 0=GPR_X 1=GPR_Y 2=GPR_Z 3=GPR_W Bits [3..2] Y mode select: 0=GPR_X 1=GPR_Y 2=GPR_Z 3=GPR_W Bits [5..4] Z mode select: 0=GPR_X 1=GPR_Y 2=GPR_Z 3=GPR_W
SQ_SPx_tp_fetch_simd_sel	SQ→SPx	1	TP Resource coming from: 0: SIMD0 1: SIMD1
SQ_SPx_vc_fetch_simd_sel	SQ→SPx	1	VC Resource coming from: 0: SIMD0 1: SIMD1
SQ_SPx_simd1_gpr_wr_addr	SQ→SPx	7	Write address

SQ_SPx_simd1_gpr_rd_addr	SQ→SPx	7	Read address
SQ_SPx_simd1_gpr_rd_en	SQ→SPx	1	Read Enable
SQ_SP0_simd1_gpr_pspv_wr_en	SQ→SP0 (SP1)	4	Write Enable for the GPRs of SP0-1 for PS and PV
SQ_SP2_simd1_gpr_pspv_wr_en	SQ→SP2 (SP3)	4	Write Enable for the GPRs of SP2-3 for PS and PV
SQ_SP4_simd1_gpr_pspv_wr_en	SQ→SP4 (SP5)	4	Write Enable for the GPRs of SP4-5 for PS and PV
SQ_SP6_simd1_gpr_pspv_wr_en	SQ→SP6 (SP7)	4	Write Enable for the GPRs of SP6-7 for PS and PV
SQ_SPx__simd1_channel_mask	SQ→SPx	4	The channel mask for SIMD1
SQ_SPx_simd1_fetch_swizzle	SQ→SPx	6	Swizzle code for the TP request (2 bits per channel ignore W as it is not used). Bits [1..0] X mode select: 0=GPR_X 1=GPR_Y 2=GPR_Z 3=GPR_W Bits [3..2] Y mode select: 0=GPR_X 1=GPR_Y 2=GPR_Z 3=GPR_W Bits [5..4] Z mode select: 0=GPR_X 1=GPR_Y 2=GPR_Z 3=GPR_W
SQ_SP0_simd1_gpr_int_wr_en	SQ→SP0	1	Write Enable for the GPRs of SP0-1 for Inputs (interp/vtx)
SQ_SP2_simd1_gpr_int_wr_en	SQ→SP2	1	Write Enable for the GPRs of SP2-3 for Inputs (interp/vtx)
SQ_SP4_simd1_gpr_int_wr_en	SQ→SP4	1	Write Enable for the GPRs of SP4-5 for Inputs (interp/vtx)
SQ_SP6_simd1_gpr_int_wr_en	SQ→SP6	1	Write Enable for the GPRs of SP6-7 for Inputs (interp/vtx)

23.2.13 *SQ to SPx:*

Name	Direction	Bits	Description
SQ_SPx_instr_start	SQ→SPx	1	Instruction start
SQ_SPx_simd0_instruct	SQ→SPx	24	Transferred over 4 cycles 0: SRC A Negate Argument Modifier 0:0 SRC A Abs Argument Modifier 1:1 SRC A Swizzle 9:2 Vector Dst 15:10 Per channel Select 23:16 00: GPR 01: PV 10: PS 11: Constant (if 11 has to be 11 for all channels) --- 1: SRC B Negate Argument Modifier 0:0 SRC B Abs Argument Modifier 1:1 SRC B Swizzle 9:2 Scalar Dst 15:10 Per channel Select 23:16 00: GPR 01: PV 10: PS 11: Constant (if 11 has to be 11 for all channels) --- 2: SRC C Negate Argument Modifier 0:0 SRC C Abs Argument Modifier 1:1 SRC C Swizzle 9:2 Unused 15:10 Per channel Select 23:16 00: GPR 01: PV 10: PS 11: Constant (if 11 has to be 11 for all channels) --- 3: Vector Opcode 4:0 Scalar Opcode 10:5 Vector Clamp 11:11 Scalar Clamp 12:12 Vector Write Mask 16:13 Scalar Write Mask 20:17 Unused 23:21
SQ_SP0_simd0_pred_override	SQ→SP0 (SP1)	4	SP0 receives 2 bits and SP1 two bits as well. 0: Use per channel RGBA field (enables the per channel logic). 1: Use GPR for PV or PS settings. LET the 11 (constant) go thru unchanged
SQ_SP2_simd0_pred_override	SQ→SP2 (SP3)	4	0: Use per channel RGBA field (enables the per channel logic). 1: Use GPR for PV or PS settings. LET the 11 (constant) go thru unchanged
SQ_SP4_simd0_pred_override	SQ→SP4 (SP5)	4	0: Use per channel RGBA field (enables the per channel logic). 1: Use GPR for PV or PS settings. LET the 11 (constant) go thru unchanged

SQ_SP6_simd0_pred_override	SQ→SP6 (SP7)	4	0: Use per channel RGBA field (enables the per channel logic). 1: Use GPR for PV or PS settings. LET the 11 (constant) go thru unchanged
SQ_SPx_simd0_stall	SQ→SPx	1	Stall signal
SQ_SPx_simd1_instruct	SQ→SPx	24	Transferred over 4 cycles 0: SRC A Negate Argument Modifier 0:0 SRC A Abs Argument Modifier 1:1 SRC A Swizzle 9:2 Vector Dst 15:10 Per channel Select 23:16 00: GPR 01: PV 10: PS 11: Constant (if 11 has to be 11 for all channels) --- 1: SRC B Negate Argument Modifier 0:0 SRC B Abs Argument Modifier 1:1 SRC B Swizzle 9:2 Scalar Dst 15:10 Per channel Select 23:16 00: GPR 01: PV 10: PS 11: Constant (if 11 has to be 11 for all channels) --- 2: SRC C Negate Argument Modifier 0:0 SRC C Abs Argument Modifier 1:1 SRC C Swizzle 9:2 Unused 15:10 Per channel Select 23:16 00: GPR 01: PV 10: PS 11: Constant (if 11 has to be 11 for all channels) --- 3: Vector Opcode 4:0 Scalar Opcode 10:5 Vector Clamp 11:11 Scalar Clamp 12:12 Vector Write Mask 16:13 Scalar Write Mask 20:17 Unused 23:21
SQ_SP0_simd0_pred_override	SQ→SP0 (SP1)	4	SP0 receives 2 bits and SP1 two bits as well. 0: Use per channel RGBA field (enables the per channel logic). 1: Use GPR for PV or PS settings. LET the 11 (constant) go thru unchanged
SQ_SP2_simd0_pred_override	SQ→SP2 (SP3)	4	0: Use per channel RGBA field (enables the per channel logic). 1: Use GPR for PV or PS settings. LET the 11 (constant) go thru unchanged
SQ_SP4_simd0_pred_override	SQ→SP4 (SP5)	4	0: Use per channel RGBA field (enables the per channel logic). 1: Use GPR for PV or PS settings. LET the 11 (constant) go thru unchanged

SQ_SP6_simd0_pred_override	SQ→SP6 (SP7)	4	0: Use per channel RGBA field (enables the per channel logic). 1: Use GPR for PV or PS settings. LET the 11 (constant) go thru unchanged
SQ_SPx_simd1_stall	SQ→SPx	1	Stall signal
SQ_SPx_export_simd_sel	SQ->SPx	1	Which SIMD engine is exporting.

23.2.14 *SQ to SX: write mask interface (must be aligned with the SP data)*

Name	Direction	Bits	Description
SQ_SX0_write_mask	SQ→SX0	8	Result of pixel kill in the shader pipe, which must be output for all pixel exports (depth and all color buffers). 4x4 because 16 pixels are computed per clock. This is for the data coming of SP0 and SP2.
SQ_SX1_ write_mask	SQ→SX1	8	Result of pixel kill in the shader pipe, which must be output for all pixel exports (depth and all color buffers). 4x4 because 16 pixels are computed per clock. This is for the data coming of SP1 and SP3.
SQ_SXx_channel_mask	SQ->SXx	4	This is the per channel export mask. It is computed by doing vector_mask | scalar_mask | bit 14 of the alu instruction.
SQ_SX0_kill_mask	SQ->SX0	8	These are the valid bits coming straight from the reservation stations.
SQ_SX1_kill_mask	SQ->SX1	8	These are the valid bits coming straight from the reservation stations.

23.2.15 *SP to SQ: Constant address load/ Predicate Set/Kill set*

Name	Direction	Bits	Description
SP0_SQ_simd0_const_addr	(SP1) SP0→SQ	36	Constant address load 18 bits from SP0 and 18 from SP4.
SP0_SQ_simd0_valid	SP0→SQ	1	Data valid
SP2_SQ_simd0_const_addr	(SP3) SP2→SQ	36	Constant address load
SP2_SQ_simd0_valid	SP2→SQ	1	Data valid
SP4_SQ_simd0_const_addr	(SP5) SP4→SQ	36	Constant address load
SP4_SQ_simd0_valid	SP4→SQ	1	Data valid
SP6_SQ_simd0_const_addr	(SP7) SP6→SQ	36	Constant address load
SP6_SQ_simd0_valid	SP6→SQ	1	Data valid
SP0_SQ_simd0_pred_kill_vector	(SP1) SP0→SQ	4	Data (predicates or kill/mask) 2 bits from SP0 and 2 bits from SP4
SP0_SQ_simd0_pred_kill_valid	SP0->SQ	1	Data valid
SP0_SQ_simd0_pred_kill_type	SP0->SQ	1	0: predicate vector 1: kill/mask vector
SP2_SQ_simd0_pred_kill_vector	(SP3) SP2→SQ	4	Data (predicates or kill/mask)
SP2_SQ_simd0_pred_kill_valid	SP2->SQ	1	Data valid
SP2_SQ_simd0_pred_kill_type	SP2->SQ	1	0: predicate vector 1: kill/mask vector
SP4_SQ_simd0_pred_kill_vector	(SP5) SP4→SQ	4	Data (predicates or kill/mask)
SP4_SQ_simd0_pred_kill_valid	SP4->SQ	1	Data valid
SP4_SQ_simd0_pred_kill_type	SP4->SQ	1	0: predicate vector 1: kill/mask vector
SP6_SQ_simd0_pred_kill_vector	(SP7) SP6→SQ	4	Data (predicates or kill/mask)
SP6_SQ_simd0_pred_kill_valid	SP6->SQ	1	Data valid
SP6_SQ_simd0_pred_kill_type	SP6->SQ	1	0: predicate vector 1: kill/mask vector
SP0_SQ_simd1_const_addr	(SP1) SP0→SQ	36	Constant address load 18 bits from SP0 and 18 from SP4.
SP0_SQ_simd1_valid	SP0→SQ	1	Data valid
SP2_SQ_simd1_const_addr	(SP3) SP2→SQ	36	Constant address load
SP2_SQ_simd1_valid	SP2→SQ	1	Data valid
SP4_SQ_simd1_const_addr	(SP5) SP4→SQ	36	Constant address load
SP4_SQ_simd1_valid	SP4→SQ	1	Data valid
SP6_SQ_simd1_const_addr	(SP7) SP6→SQ	36	Constant address load
SP6_SQ_simd1_valid	SP6→SQ	1	Data valid
SP0_SQ_simd1_pred_kill_vector	(SP1) SP0→SQ	4	Data (predicates or kill/mask) 2 bits from SP0 and 2 bits from SP4
SP0_SQ_simd1_pred_kill_valid	SP0->SQ	1	Data valid
SP0_SQ_simd1_pred_kill_type	SP0->SQ	1	0: predicate vector 1: kill/mask vector
SP2_SQ_simd1_pred_kill_vector	(SP3) SP2→SQ	4	Data (predicates or kill/mask)
SP2_SQ_simd1_pred_kill_valid	SP2->SQ	1	Data valid
SP2_SQ_simd1_pred_kill_type	SP2->SQ	1	0: predicate vector 1: kill/mask vector
SP4_SQ_simd1_pred_kill_vector	(SP5) SP4→SQ	4	Data (predicates or kill/mask)
SP4_SQ_simd1_pred_kill_valid	SP4->SQ	1	Data valid
SP4_SQ_simd1_pred_kill_type	SP4->SQ	1	0: predicate vector 1: kill/mask vector
SP6_SQ_simd1_pred_kill_vector	(SP7) SP6→SQ	4	Data (predicates or kill/mask)
SP6_SQ_simd1_pred_kill_valid	SP6->SQ	1	Data valid
SP6_SQ_simd1_pred_kill_type	SP6->SQ	1	0: predicate vector 1: kill/mask vector

Because of the sharing of the bus none of the MOVA, PREDSET or KILL instructions may be coissued.

23.2.16 *SQ to SPx: constant broadcast*

Name	Direction	Bits	Description

SQ_SPx_simd0_const	SQ→SPx	128	Constant broadcast
SQ_SPx_simd1_const	SQ→SPx	128	Constant broadcast
SQ_SPx_simd0_const_sel	SQ→SPx	2	Use the incoming constant instead of the registered one for the next group of 16. 0 : Normal mode 1: Waterfall on SRCA 2: Waterfall on SRCB 3: Waterfall on SRCC
SQ_SPx_simd1_const_sel	SQ→SPx	2	Use the incoming constant instead of the registered one for the next group of 16. 0 : Normal mode 1: Waterfall on SRCA 2: Waterfall on SRCB 3: Waterfall on SRCC

23.2.17 *SQ to CP: RBBM bus*

Name	Direction	Bits	Description
SQ_RBB_rs	SQ→CP	1	Read Strobe
SQ_RBB_rd	SQ→CP	32	Read Data
SQ_RBBM_nrtrtr	SQ→CP	1	Optional
SQ_RBBM_rtr	SQ→CP	1	Real-Time (Optional)

23.2.18 *CP to SQ: RBBM bus*

Name	Direction	Bits	Description
rbbm_we	CP→SQ	1	Write Enable
rbbm_a	CP→SQ	15	Address -- Upper Extent is TBD (16:2)
rbbm_wd	CP→SQ	32	Data
rbbm_be	CP→SQ	4	Byte Enables
rbbm_re	CP→SQ	1	Read Enable
rbb_rs0	CP→SQ	1	Read Return Strobe 0
rbb_rs1	CP→SQ	1	Read Return Strobe 1
rbb_rd0	CP→SQ	32	Read Data 0
rbb_rd1	CP→SQ	32	Read Data 0
RBBM_SQ_soft_reset	CP→SQ	1	Soft Reset

23.2.19 *SQ to CP: State report*

Name	Direction	Bits	Description
SQ_CP_vs_event	SQ→CP	1	Vertex Shader Event
SQ_CP_vs_eventid	SQ→CP	5	Vertex Shader Event ID
SQ_CP_ps_event	SQ→CP	1	Pixel Shader Event
SQ_CP_ps_eventid	SQ→CP	5	Pixel Shader Event ID

23.3 Example of control flow program execution

We now provide some examples of execution to better illustrate the new design.

Given the program:

Alu 0
Alu 1
Tex 0
Tex 1
Alu 3 Serial
Alu 4
Tex 2
Alu 5
Alu 6 Serial

Tex 3
Alu 7
Alloc Position 1 buffer
Alu 8 Export
Tex 4
Alloc Parameter 3 buffers
Alu 9 Export 0
Tex 5
Alu 10 Serial Export 2
Alu 11 Export 1 End

Would be converted into the following CF instructions:

```
Execute 0 Alu 0 Alu 0 Tex 0 Tex 1 Alu 0 Alu 0 Tex 0 Alu 1 Alu 0 Tex
Execute 0 Alu
Alloc Position 1
Execute 0 Alu 0 Tex
Alloc Param 3
Execute_end 0 Alu 0 Tex 1 Alu 0 Alu
```

And the execution of this program would look like this:

Put thread in Vertex RS:

Control Flow Instruction Pointer (12 bits), (CFP)
Execution Count Marker (3 or 4 bits), (ECM)
Loop Iterators (4x9 bits), (LI)
Call return pointers (4x12 bits), (CRP)
Predicate Bits(4x64 bits), (PB)
Export ID (1 bit), (EXID)
GPR Base Ptr (8 bits), (GPR)
Export Base Ptr (7 bits), (EB)
Context Ptr (3 bits).(CPTR)
LOD correction bits (16x6 bits) (LOD)

State Bits									
CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
0	0	0	0	0	0	0	0	0	0

Valid Thread (VALID)
Texture/ALU engine needed (TYPE)
Texture Reads are outstanding (PENDING)
Waiting on Texture Read to Complete (SERIAL)
Allocation Wait (2 bits) (ALLOC)
- 00 – No allocation needed
- 01 – Position export allocation needed (ordered export)
- 10 – Parameter or pixel export needed (ordered export)
- 11 – pass thru (out of order export)

Allocation Size (4 bits) (SIZE)
Position Allocated (POS_ALLOC)
First thread of a new context (FIRST)
Last (1 bit), (LAST)

Status Bits								
VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	ALU	0	0	0	0	0	1	0

Then the thread is picked up for the execution of the first control flow instruction:

```
Execute 0 Alu 0 Alu 0 Tex 0 Tex 1 Alu 0 Alu 0 Tex 0 Alu 1 Alu 0 Tex
```

It executes the first two ALU instructions and goes back to the RS for a resource request change. Here is the state returned to the RS:

State Bits

CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
0	2	0	0	0	0	0	0	0	0

Status Bits

VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	TEX	0	0	0	0	0	1	0

Then when the texture pipe frees up, the arbiter picks up the thread to issue the texture reads. The thread comes back in this state:

State Bits

CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
0	4	0	0	0	0	0	0	0	0

Status Bits

VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	ALU	1	1	0	0	0	1	0

Because of the serial bit the arbiter must wait for the texture to return and clear the PENDING bit before it can pick the thread up. Lets say that the texture reads are complete, then the arbiter picks up the thread and returns it in this state:

State Bits

CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
0	6	0	0	0	0	0	0	0	0

Status Bits

VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	TEX	0	0	0	0	0	1	0

Again the TP frees up, the arbiter picks up the thread and executes. It returns in this state:

State Bits

CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
0	7	0	0	0	0	0	0	0	0

Status Bits

VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	ALU	1	0	0	0	0	1	0

Now, even if the texture has not returned we can still pick up the thread for ALU execution because the serial bit is not set. The thread will however come back to the RS for the second ALU instruction because it has the serial bit set.

State Bits

CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
0	8	0	0	0	0	0	0	0	0

Status Bits

VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	ALU	1	1	0	0	0	1	0

As soon as the TP clears the pending bit the thread is picked up and returns:

State Bits

CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
0	9	0	0	0	0	0	0	0	0

Status Bits

VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	TEX	0	0	0	0	0	1	0

Picked up by the TP and returns:

```
Execute 0 Alu
```

State Bits

CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
1	0	0	0	0	0	0	0	0	0

Status Bits

VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	ALU	1	0	0	0	0	1	0

Picked up by the ALU and returns (lets say the TP has not returned yet):

```
Alloc Position 1
```

State Bits

CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
2	0	0	0	0	0	0	0	0	0

Status Bits

VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	ALU	1	0	01	1	0	1	0

If the SX has the place for the export, the SQ is going to allocate and pick up the thread for execution. It returns to the RS in this state:

```
Execute 0 Alu 0 Tex
```

State Bits

CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
3	1	0	0	0	0	0	0	0	0

Status Bits								
VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	TEX	1	0	0	0	1	1	0

Now, since the TP has not returned yet, we must wait for it to return because we cannot issue multiple texture requests. The TP returns, clears the PENDING bit and we proceed:

```
Alloc Param 3
```

State Bits									
CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
4	0	0	0	0	1	0	0	0	0

Status Bits								
VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	ALU	1	0	10	3	1	1	0

Once again the SQ makes sure the SX has enough room in the Parameter cache before it can pick up this thread.

```
Execute_end 0 Alu 0 Tex 1 Alu 0 Alu
```

State Bits									
CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
5	1	0	0	0	1	0	100	0	0

Status Bits								
VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	TEX	1	0	0	0	1	1	0

This executes on the TP and then returns:

State Bits									
CFP	ECM	LI	CRP	PB	EXID	GPR	EB	CPTR	LOD
5	2	0	0	0	1	0	100	0	0

Status Bits								
VALID	TYPE	PENDING	SERIAL	ALLOC	SIZE	POS_ALLOC	FIRST	LAST
1	ALU	1	1	0	0	1	1	1

Waits for the TP to return because of the textures reads are pending (and SERIAL in this case). Then executes and does not return to the RS because the LAST bit is set. This is the end of this thread and before dropping it on the floor, the SQ notifies the SX of export completion.

24. Open issues

Need to do some testing on the size of the register file as well as on the register file allocation method (dynamic VS static).

Saving power?